

WebAssembly 入門

2026-10-03 | セキュリティ・キャンプ 2026 ミニ（鹿児島開催）
わたすけ (@watasuke1024)



渡辺 耀介 WATANABE Yosuke

- インターネットで **わたすけ** (@watasuke1024) として生活しています
- 宇部高専 制御情報工学科 (～2025.03)
→ 筑波大学 情報メディア創成学類 (**3 年次編入**)
- 専攻分野：ヒューマンコンピュータインタラクション
 - UI とかインタフェースとかそのへん
 - 僕は **xR** (AR/VR 等) **による作業環境**について研究中
- 趣味：**ソフトウェア開発**、作曲、料理



渡辺 耀介 WATANABE Yosuke

- インターネットで **わたすけ** (@watasuke1024) として生活しています
- 宇部高専 制御情報工学科 (～2025.03)
→ 筑波大学 情報メディア創成学類 (**3 年次編入**)
- 専攻分野：ヒューマンコンピュータインタラクション
 - UI とかインタフェースとかそのへん
 - 僕は **xR** (AR/VR 等) **による作業環境** について研究中
- 趣味：**ソフトウェア開発**、作曲、料理

人生の目標：可搬性の高い環境を作りたい

pkm.watasuke.net

検索

Explorer

notes

- 1週間の振り返り
- 9P protocol
- 2022年度 情報処理学会関西支部 記念講演会
- a11y
- ADT
- AfterEffects
- aptとapt-getの違い
- Arch Linuxインストール録
- Arch LinuxでSteamVR
- BMI
- BNF
- C++のクラス
- C++のムーブセマンティクス
- Chromiumがbindする
- Waylandプロトコル
- ChromiumのURL オープン処理
- clang

可搬性の高い環境を作る

Properties 1

tags #ポエム #夢・将来像? #hci

2026年4月09日, 7 min read

- 人間のやることを減らすために必要
 - とか、アプローチの一つ
- Dockerなどを用いた開発「環境」も、まあ広い意味では含まれると思う
- 環境の可搬性が高いと何がうれしい？
 - 通学中でも入浴中でも、いつでも即座に情報にアクセスすることができる
 - やるべきことを思い出して、わざわざスマホを取り出してToDoアプリを開きに行く？ 忘れたらどうするんですか
 - 座席が空いていない電車で作業ができて、うれしい
 - 実は立ってるとノートパソコンを快適に触れなくて……
 - 複数の端末で同等の環境を構築する必要がない
 - デスクトップのWindowsにも、ノートPCのLinuxにもVSCodeを入れて、それぞれGitHubアカウントでログインして同期をONにして～ ← ダルい
 - VSCodeは同期機能があるからまだいいけど
 - NixOSで解決……？
 - 環境が同期されたとしても、作業状況（編集中のファイル、開かれているウィンドウ）の状態が同期されていないと、作業の再開はできない
 - 仮にSyncthingなり何なりを使ってファイル等を同期できるようにしたとしても、同期にはインターネット接続が必要で、通信容量の消費や速度低下による体験の悪化が発生してしまいそう

pkm.watasuke.net

どこでも作業したい！

- 中学生時代：
 - 電子辞書でメモ帳を起動してプログラムを書く
 - パソコン部で使うソフトを USB に入れて持ち運び
- 高専時代～
 - サーバーを契約して、そこに SSH して全ての開発をやる
 - スペックが低くて無理だった
 - 自宅サーバーに VPN 経由で SSH
 - 今はこれがち

ところで

ところで

- 高専 1 年生の冬、英単語テストに苦しめられていた

ところで

- 高専 1 年生の冬、英単語テストに苦しめられていた
- 単語帳アプリにインポートするための csv を作って Google ドライブに上げてクラスメイトに共有していた

ところで

- 高専 1 年生の冬、英単語テストに苦しめられていた
- 単語帳アプリにインポートするための csv を作って Google ドライブに上げてクラスメイトに共有していた
- すると、typo や抜けが頻繁に発生する

ところで

- 高専 1 年生の冬、英単語テストに苦しめられていた
- 単語帳アプリにインポートするための csv を作って Google ドライブに上げてクラスメイトに共有していた
- すると、typo や抜けが頻繁に発生する
- すでに csv をインポートしている人は手作業で修正

ところで

- 高専 1 年生の冬、英単語テストに苦しめられていた
- 単語帳アプリにインポートするための csv を作って Google ドライブに上げてクラスメイトに共有していた
- すると、typo や抜けが頻繁に発生する
- すでに csv をインポートしている人は手作業で修正
- 面倒だから英単語の一問一答サービスでも作るか！ w

Web アプリ、すばらしい

- クラスメイトのほとんどは iPhone ユーザ
- 僕は iOS アプリの開発に必要な macOS を使えない
- ほな Web アプリか……

Web アプリ、すばらしい

- クラスメイトのほとんどは iPhone ユーザ
- 僕は iOS アプリの開発に必要な macOS を使えない
- ほな Web アプリか……

→ ちょっと CSS を頑張るだけで**スマホの OS どころか
PC/スマホという機種差すら無視してアプリを作れる！**

どこでも使える（家では PC、外出先ではスマホ）
== 可搬性が高いのでは！？

Web アプリ、すばらしい

- なので、Web を前提としてプログラミング言語を作った
 - コンパイル結果が WebAssembly
- Rust で処理系を書いたので
コンパイラ自体も WebAssembly として出力できる
- Web ブラウザで簡単に言語を試せる！
 - コンパイルから実行まですべて Web ブラウザで行える

input Run (Ctrl+Enter)

```
/*
  Settlang minimal example
*/
*#
fn accumulator(self: i64, e: i64) -> i64 {
  # just return added value
  ret self + e
}

fn randomize() -> i64 {
  ret random(0, 100)
}

# function named `main` is entrypoint
fn main() -> i64 {
  # they have 'setter' function, so mutable
  let sum: i64 | accumulator = 0
  let rnd: i64 | randomize = 0
  for i in 0..10 {
    sum = accumulator sum rnd
    rnd = randomize()
  }
  ret sum
}
```

Examples Default FizzBuzz GuessNumber Setter Forloop

output

```
55, 75, 70, 78, 59, 59, 68, 97, 65, 13
result=639
```

Settlang

watasuke102.github.io/settlang

Settlang - 変数の可変性を setter の有無により表現する言語

Web 技術、すばらしい

↑と思ったので WebAssembly の講義をやりま
す
よろしくお願いします

講義のめあて

1. WebAssembly という切り口から **Web 技術**に触れる
2. **バイナリ**という概念を知って、親しみを持つ
3. 「**仕様書を読む**」という選択肢を思い浮かべられるようになる

目次

1. WebAssembly とは
2. Wasm テキスト形式
3. もっと Wasm を書いてみる
4. Wasm をブラウザで実行してみよう

目次

1. WebAssembly とは
2. Wasm テキスト形式
3. もっと Wasm を書いてみる
4. Wasm をブラウザで実行してみよう

WebAssembly とは

- Web ブラウザ**等**で動作するもの
- ポータブルさを意識した仕様策定が行われている
 - Web ブラウザに限らず様々な環境で動く
- **サンドボックス環境で動作**するため安全
 - 明示的に与えたものにしかアクセスできない



WebAssembly とは

- 略して **Wasm**

- 公式サイト¹にも書いてある：

- WebAssembly** (abbreviated **Wasm**)

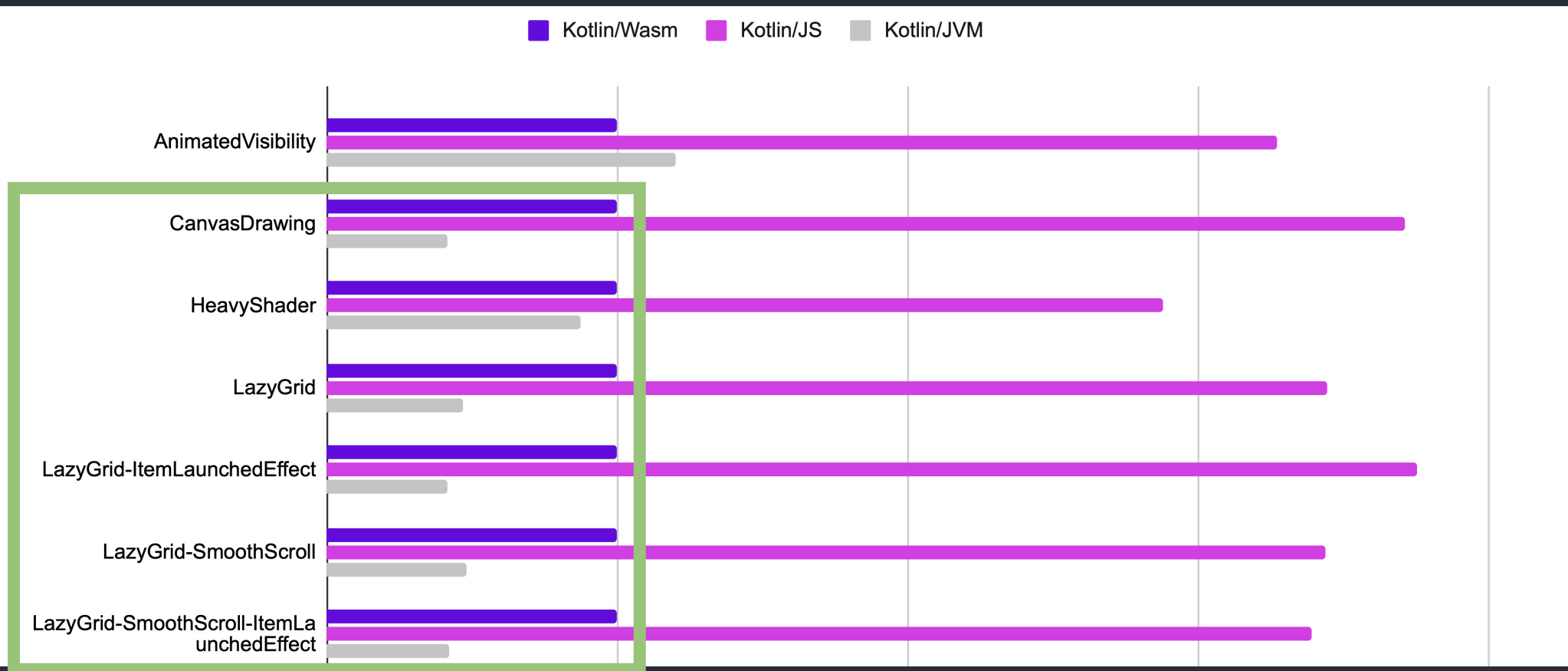
- WASM ではない！



¹ <https://webassembly.org>

Wasm は速い

本当に？



Kotlin/JVM のほうが速い (バーが短いほどかかった時間が短い)

Wasm は速い

本当に？ → 部分的に**いいえ**

Wasm にしたから絶対に速くなる！ というわけでもない

じゃあなぜ Wasm ?

様々な言語から出力できる

- ポータブルな仕様のおかげ
- C++ (Emscripten), Rust, Go, C#, Kotlin, MoonBit, Java...²
- 自分が慣れている言語で Web 開発ができる

² <https://github.com/appcypher/awesome-wasm-langs> にたくさん載っている

じゃあなぜ Wasm ?

様々な言語から出力できる

- プラグインとして Wasm をサポートすると、
開発者に言語を強制しなくてよくなる
 - 例) Typst は Wasm 製プラグインをサポートしている
 - 言語それぞれに SDK を用意する必要がない！

応募課題

WebAssembly 形式にコンパイルできるプログラミング言語を 1 つ選んで、以下に取り組んでください。

- (2) 選んだプログラミング言語を用いて「**引数として数字を 2 つ受け取って、それらを足した数を返す**」という関数を作ってください。書いたプログラムを答えてください。
- (3) (2) で作成した関数を使って **1+2 を計算した結果を、標準出力に出力するプログラム**を書いてください。そして、そのプログラムを WebAssembly 形式で出力してみてください。

応募課題

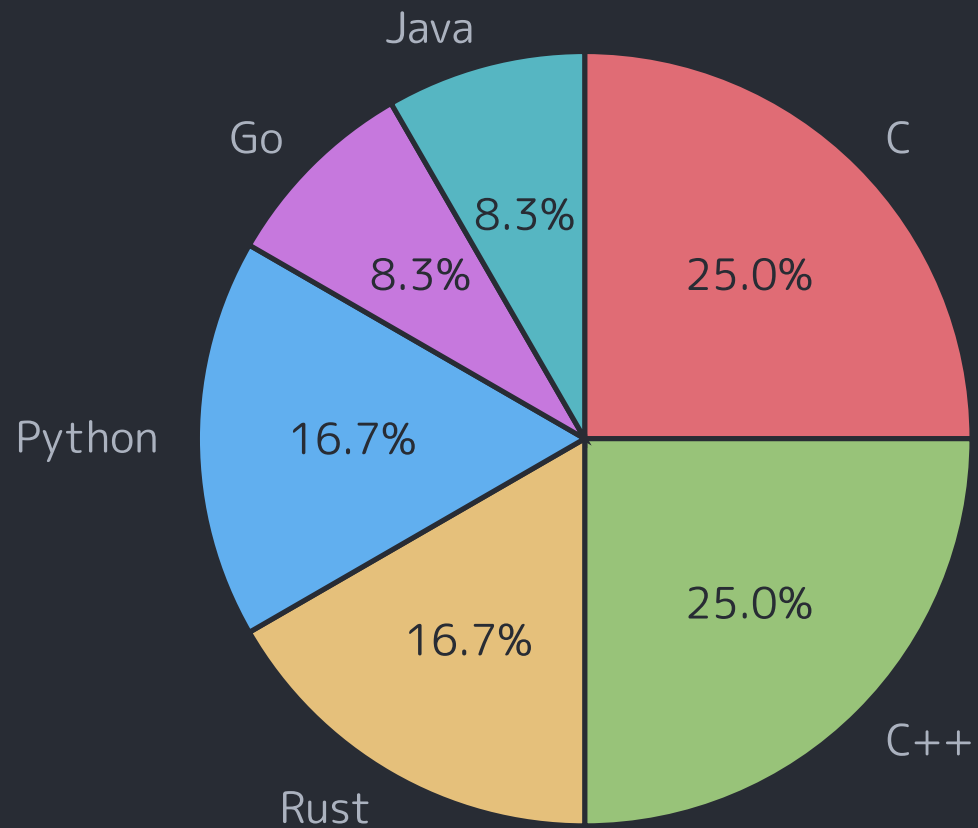
WebAssembly 形式にコンパイルできるプログラミング言語を

1 つ選んで、以下に取り組んでください。

(2) 選んだプログラミング言語を用いて「**引数として数字を 2 つ受け取って、それらを足した数を返す**」という関数を作ってください。書いたプログラムを答えてください。

(3) (2) で作成した関数を使って **1+2 を計算した結果を、標準出力に出力するプログラム**を書いてください。そして、そのプログラムを WebAssembly 形式で出力してみてください。

事前課題にて使われた言語の割合



想定回答

C++

```
int32_t add(int32_t lhs, int32_t rhs) {  
    return lhs + rhs;  
}  
  
int main() {  
    std::println("{} ", add(1, 2));  
}
```

※ lhs = Left Hand Side

想定回答

Rust (再掲)

```
fn add(lhs: i32, rhs: i32) -> i32 {  
    lhs + rhs  
}  
  
fn main() {  
    println!("{}", add(1, 2));  
}
```

int32_t / i32 って何？

符号付き 32bit 整数のこと

32 個の 0/1 (bit) により整数を表す

マイナスも表現できる

符号なし 64bit 整数というものもある (uint64_t, u64)

符号付き

符号なし

16bit	$-32,768 \sim 32,767$	$0 \sim 65,535$
32bit	$-2,147,483,648 \sim$ $2,147,483,647$	$0 \sim 4,294,967,295$
64bit	$-9,223,372,036,854,775,808 \sim$ $9,223,372,036,854,775,807$	$0 \sim 18,446,744,073,709,551,615^3$

3 1,844 京 6,744 兆 737 億 955 万 1,615

2 進数とビット/バイト

- 我々がよく使うのは 10 進数
 - それぞれの位は 0~9 によって表される
 - 右から N (≥ 0) 番目の位に 10^N をかけて足すと値が得られる

例：1234

$$\begin{aligned} 1234 &= 1 \times 10^3 + 2 \times 10^2 + 3 \times 10^1 + 4 \times 10^0 \\ &= 1000 + 200 + 30 + 4 \\ &= 1234 \end{aligned}$$

2 進数とビット/ バイト

- コンピュータでは **2 進数** が用いられる
 - それぞれの位は 0, 1 によって表される
 - 右から N (≥ 0) 番目の位に 2^N をかけて足すと値が得られる

例： **0b**010101

$$\begin{aligned} 010101 &= 0 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\ &= 0 + 16 + 0 + 4 + 0 + 1 \\ &= 21 \end{aligned}$$

2 進数とビット/ バイト

- コンピューターでは、2 進数の 1 桁を**ビット** (bit) と呼ぶ
- 多くの場合⁴、8 ビットをまとめて**1 バイト** (byte) と呼ぶ
 - 例) 32 bit == 4 byte

⁴ 昔はそうではなかったので 8 ビットのことをオクテットと呼んだりもしていたらしい

16 進数

- 2 進数は長すぎる！ → **16 進数**を用いてコンパクトに表す
 - それぞれの位は 0~9 と a~f の 16 文字によって表される
 - 右から $N (\geq 0)$ 番目の位に 16^N をかけて足す
- 1 バイトをちょうど 2 桁で表せて、うれしい

例： **0x**1af ($a == 10, f == 15$)

$$\begin{aligned} 1af &= 1 \times 16^2 + 10 \times 16^1 + 15 \times 16^0 \\ &= 256 + 160 + 15 \\ &= 431 \end{aligned}$$

想定回答

Rust (再掲)

```
fn add(lhs: i32, rhs: i32) -> i32 {  
    lhs + rhs  
}  
  
fn main() {  
    println!("{}", add(1, 2));  
}
```

想定回答

Rust

```
$ rustc --target wasm32-wasip2 add.rs  
$ wasmtime add.wasm  
3
```

想定回答

Rust

```
$ rustc --target wasm32-wasip2 add.rs
```

```
$ wasmtime add.wasm
```

```
3          ↑ これ何？
```

file コマンド

ファイルがどういうものなのか教えてくれる

```
$ file /usr/bin/ls
/usr/bin/ls: ELF 64-bit LSB pie executable, x86-64,
version 1 (SYSV), dynamically linked, interpreter /
lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=4e297d3b427342e1da6b66f5ca0fd279f43f3afe,
for GNU/Linux 4.4.0, stripped
$ file add.rs
add.rs: C source, ASCII text
```

add.wasm の形式

```
$ file add.wasm
```

```
add.wasm: WebAssembly (wasm) binary version 0x1000d  
(component)
```

add.wasm の形式

```
$ file add.wasm
```

```
add.wasm: WebAssembly (wasm) binary version 0x1000d  
(component)
```

Wasm とは「本当は」何なのか

WebAssembly (abbreviated Wasm) is a
binary instruction format for a stack-based virtual machine

バイナリの命令フォーマット

どうということ？

こうということ⁵

```
$ od -tx1 add.wasm | head -n5
00000000 00 61 73 6d 0d 00 01 00 07 24 01 42 04 01 6f 02
00000020 73 73 01 70 00 01 40 00 00 01 04 00 0f 67 65 74
00000040 2d 65 6e 76 69 72 6f 6e 6d 65 6e 74 01 02 0a 20
00000060 01 00 1b 77 61 73 69 3a 63 6c 69 2f 65 6e 76 69
00000100 72 6f 6e 6d 65 6e 74 40 30 2e 32 2e 31 32 05 00
```

⁵ od はファイルの中身を指定した形式により出力するコマンド。

-t はフォーマットを指定するもので、x1 とすると hex (16 進数) を 1 つずつ表示する

Wasm バイナリ形式

- ブラウザ等に読み込ませて実行するもの
 - 人間が読むことを想定していない
- わかりづらすぎるので**テキスト形式**というものもある

目次

1. WebAssembly とは
2. Wasm テキスト形式
3. もっと Wasm を書いてみる
4. Wasm をブラウザで実行してみよう

はじめる前に

環境構築はしてきましたか？

環境構築 (Windows 11)

1. スタートボタン (画面端のタスクバーにある Windows ロゴ) を右クリック
2. “ターミナル” (もしくは “PowerShell”) と書かれた項目をクリック
3. “ターミナルに入力→Enter” を 1 行ずつやる：

```
cd  
pacman -S --noconfirm git wasm-tools wasmtime npm nodejs  
git clone https://github.com/watasuke102/seccamp26mini-kagosima-wasm
```

はじめる前に

してきた人へ：CLI コマンドが使える環境を用意してください

事前の指示どおりに環境構築を行った人

(Windows + WSL の人) は以下のようにして起動

1. スタートボタン (画面端のタスクバーにある Windows ロゴ) を
右クリック
2. “ターミナル” (もしくは “PowerShell”) と書かれた項目をクリック
3. `wsl -d archlinux` と入力して Enter

はじめる前に

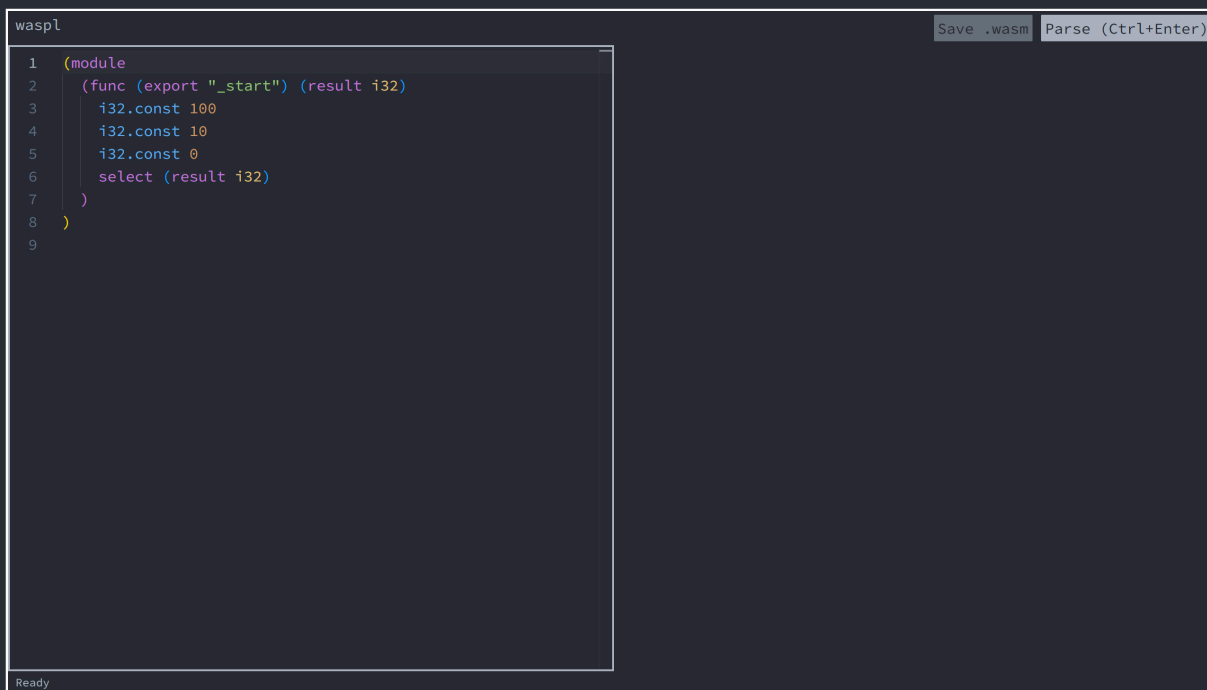
“ターミナルに入力→Enter” を 1 行ずつやる：

```
cd  
pacman -Syu --noconfirm && pacman -S git --noconfirm  
git clone https://github.com/watasuke102/seccamp26mini-kagosima-wasm
```

まずは書いてみよう

waspl という Web アプリを使います

watasuke102.github.io/waspl にアクセス！



The screenshot shows a web application titled "waspl". It features a code editor with the following code:

```
1 (module
2   (func (export "_start") (result i32)
3     i32.const 100
4     i32.const 10
5     i32.const 0
6     select (result i32)
7   )
8 )
9
```

At the top right of the editor area, there are two buttons: "Save .wasm" and "Parse (Ctrl+Enter)". At the bottom left, there is a status indicator that says "Ready".

こういうページが見える

まずは書いてみよう

画面左側がエディタになっています

1. 入力されているコードを全て消す
2. 以下の 1 行を入力：

```
(module)
```

3. Ctrl+Enter もしくは右上の「Parse」ボタンを押す

まずは書いてみよう

- こうなるはず
- 左は Wasm テキスト形式
- 右は**コンパイルされた Wasm バイナリ形式**

The screenshot shows a WebAssembly editor interface. On the left, a text editor labeled 'waspl' contains the following code:

```
1  (module)
2
```

At the bottom left of the text editor, it says '8 bytes'. On the right, there are two buttons: 'Save .wasm' and 'Parse (Ctrl+Enter)'. Below these buttons is a hex dump view showing the binary representation of the code. The hex dump has columns for 'Offset', 'Hex', and 'ASCII'. The first row shows the offset '00000000' and the hex value '00 61 73 6D 01 00 00 00', which corresponds to the ASCII string '.asm....'.

Offset	Hex	ASCII
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F	00 61 73 6D 01 00 00 00	.asm....

Wasm バイナリ形式

00 61 73 6D 01 00 00 00

何これ？

思い出し

Rust から作成した add.wasm のバイナリってどうなってたっけ

```
$ od -tx1 add.wasm | head -n5
```

```
00000000 00 61 73 6d 0d 00 01 00 07 24 01 42 04 01 6f 02
00000020 73 73 01 70 00 01 40 00 00 01 04 00 0f 67 65 74
00000040 2d 65 6e 76 69 72 6f 6e 6d 65 6e 74 01 02 0a 20
00000060 01 00 1b 77 61 73 69 3a 63 6c 69 2f 65 6e 76 69
00000100 72 6f 6e 6d 65 6e 74 40 30 2e 32 2e 31 32 05 00
```

思い出し

Rust から作成した add.wasm のバイナリってどうなってるっけ

```
$ od -tx1 add.wasm | head -n5
```

```
00000000 00 61 73 6d 0d 00 01 00 07 24 01 42 04 01 6f 02
00000020 73 73 01 70 00 01 40 00 00 01 04 00 0f 67 65 74
```

waspl

Save .wasm Parse (Ctrl+Enter)

1	(module)	Offset	00	01	02	03	04	05	06	07	08
2		00000000	00	61	73	6D	01	00	00	00	

似ているけど違う

なぜ？

00000000 00 61 73 6d 0d 00 01 00 07 24 01 42 04 01 6f 02

Offset	00	01	02	03	04	05	06	07	08
000000000	00	61	73	6D	01	00	00	00	

似ているけど違う

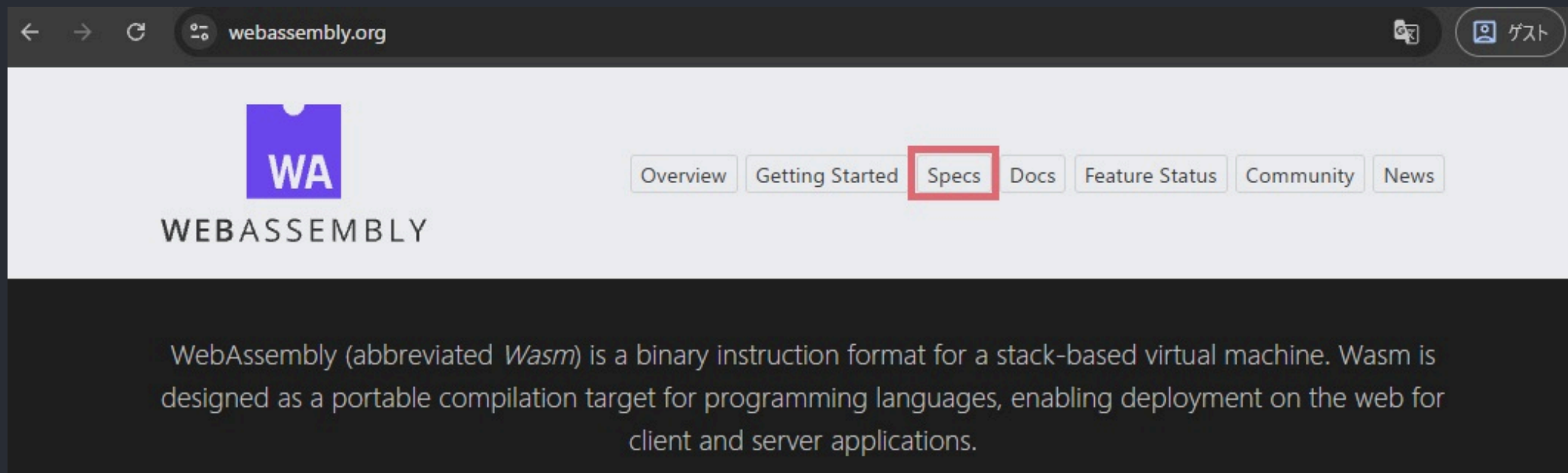
なぜ？

```
00000000 00 61 73 6d 0d 00 01 00 07 24 01 42 04 01 6f 02
```

Offset	00	01	02	03	04	05	06	07	08
000000000	00	61	73	6D	01	00	00	00	

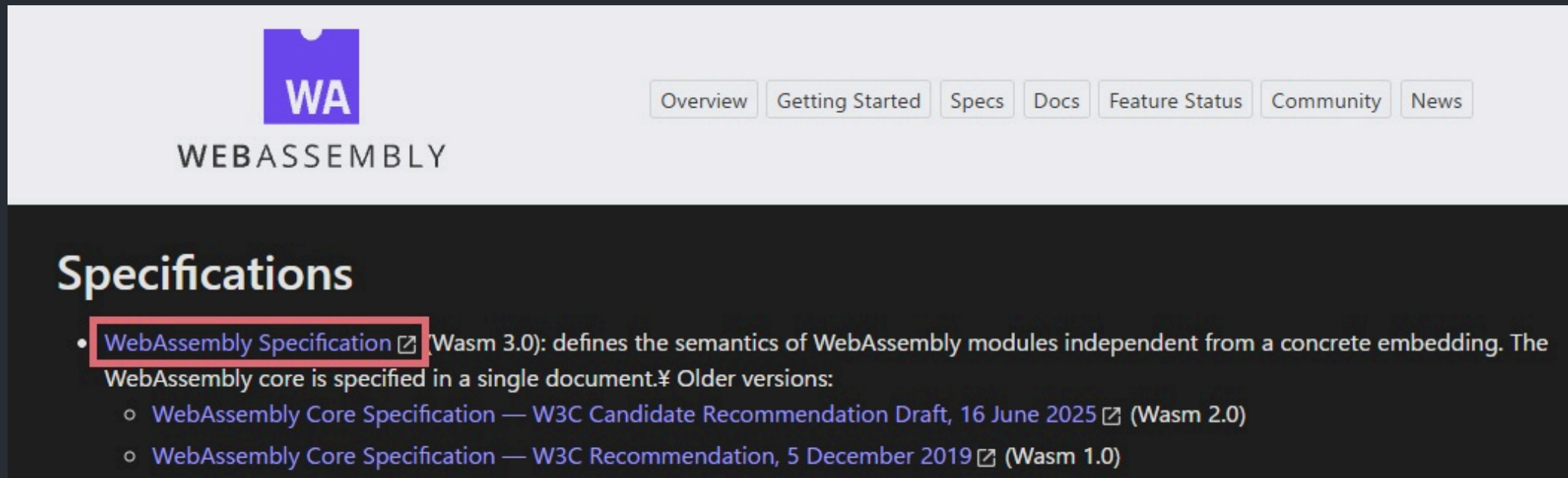
→ バイナリの構造は**仕様書**に書いてある

仕様書を読もう！



ページ上部の“**Specs**”をクリック
(SPECification == 仕様書 のこと)

仕様書を読もう！



The screenshot shows the top section of the WebAssembly website. On the left is the WebAssembly logo, which consists of a purple square with the letters 'WA' in white, and the word 'WEBASSEMBLY' in a smaller, grey font below it. To the right of the logo is a horizontal navigation bar with seven buttons: 'Overview', 'Getting Started', 'Specs', 'Docs', 'Feature Status', 'Community', and 'News'. Below the navigation bar is a dark grey section titled 'Specifications' in a large, bold, white font. Under this title is a bulleted list. The first item is 'WebAssembly Specification' with an external link icon, which is highlighted by a red rectangular box. To its right is the text '(Wasm 3.0): defines the semantics of WebAssembly modules independent from a concrete embedding. The WebAssembly core is specified in a single document.¥ Older versions:'. Below this are two sub-bullets: 'WebAssembly Core Specification — W3C Candidate Recommendation Draft, 16 June 2025' with an external link icon and '(Wasm 2.0)', and 'WebAssembly Core Specification — W3C Recommendation, 5 December 2019' with an external link icon and '(Wasm 1.0)'.

WA
WEBASSEMBLY

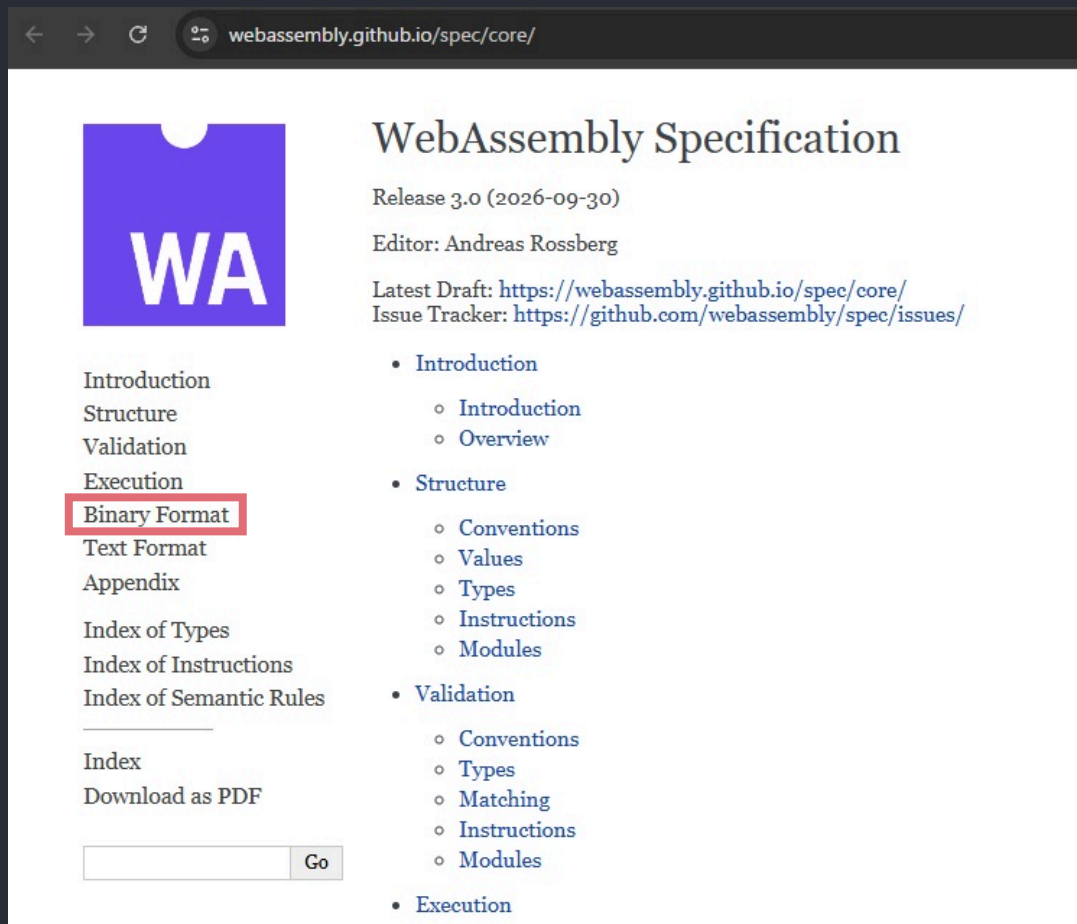
Overview Getting Started Specs Docs Feature Status Community News

Specifications

- [WebAssembly Specification](#) (Wasm 3.0): defines the semantics of WebAssembly modules independent from a concrete embedding. The WebAssembly core is specified in a single document.¥ Older versions:
 - [WebAssembly Core Specification — W3C Candidate Recommendation Draft, 16 June 2025](#) (Wasm 2.0)
 - [WebAssembly Core Specification — W3C Recommendation, 5 December 2019](#) (Wasm 1.0)

WebAssembly Specification

仕様書を読もう！



The screenshot shows the WebAssembly Specification website. The browser address bar displays 'webassembly.github.io/spec/core/'. The page features a purple logo with 'WA' on the left. The main heading is 'WebAssembly Specification', followed by 'Release 3.0 (2026-09-30)', 'Editor: Andreas Rossberg', and links for 'Latest Draft' and 'Issue Tracker'. A left sidebar lists navigation items: 'Introduction', 'Structure', 'Validation', 'Execution', 'Binary Format' (highlighted with a red box), 'Text Format', 'Appendix', 'Index of Types', 'Index of Instructions', 'Index of Semantic Rules', 'Index', and 'Download as PDF'. The main content area shows a table of contents with sections like 'Introduction', 'Structure', 'Validation', and 'Execution', each containing sub-items like 'Introduction', 'Overview', 'Conventions', 'Values', 'Types', 'Instructions', and 'Modules'.

webassembly.github.io/spec/core/

WebAssembly Specification

Release 3.0 (2026-09-30)

Editor: Andreas Rossberg

Latest Draft: <https://webassembly.github.io/spec/core/>
Issue Tracker: <https://github.com/webassembly/spec/issues/>

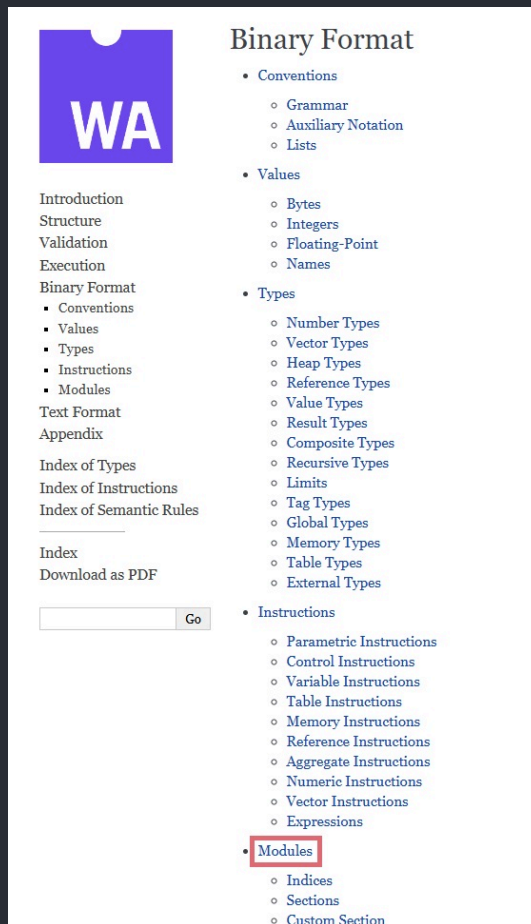
- Introduction
 - Introduction
 - Overview
- Structure
 - Conventions
 - Values
 - Types
 - Instructions
 - Modules
- Validation
 - Conventions
 - Types
 - Matching
 - Instructions
 - Modules
- Execution

Introduction
Structure
Validation
Execution
Binary Format
Text Format
Appendix
Index of Types
Index of Instructions
Index of Semantic Rules
Index
Download as PDF

Go

Binary Format (メニューに Text Format もありますね)

仕様書を読もう！



下の方にある “Modules”

仕様書を読もう！



Introduction

Structure

Validation

Execution

Binary Format

- Conventions
- Values
- Types
- Instructions

Modules

The encoding of a **module** starts with a preamble containing a 4-byte magic number (the string ‘\0asm’) and a version field. The current version of the WebAssembly binary format is 1.

The preamble is followed by a sequence of **sections**. **Custom sections** may be inserted at any place in this sequence, while other sections must occur at most once and in the prescribed order. All sections can be empty.

The lengths of lists produced by the (possibly empty) **function** and **code** section must match up.

Similarly, the optional data count must match the length of the **data segment** list. Furthermore, it must be present if any **data index** occurs in the code section.

```
magic ::= 0x00 0x61 0x73 0x6D
version ::= 0x01 0x00 0x00 0x00
module ::= magic version
         customsec* type*:typesec
         customsec* import*:importsec
```

⇒

下までスクロールするとこれが出てくる

仕様書を読もう！



Introduction

Structure

Validation

Execution

Binary Format

- Conventions
- Values
- Types
- Instructions

Modules

The encoding of a **module** starts with a preamble containing a 4-byte magic number (the string ‘\0asm’) and a version field. The current version of the WebAssembly binary format is 1.

The preamble is followed by a sequence of **sections**. **Custom sections** may be inserted at any place in this sequence, while other sections must occur at most once and in the prescribed order. All sections can be empty.

The lengths of lists produced by the (possibly empty) **function** and **code** section must match up.

Similarly, the optional data count must match the length of the **data segment** list. Furthermore, it must be present if any **data index** occurs in the code section.

```
magic ::= 0x00 0x61 0x73 0x6D
version ::= 0x01 0x00 0x00 0x00
module ::= magic version
         customsec* type*
         customsec* impo
```

← Wasm モジュールは
magic, version という**順番**で並ぶ

下までスクロールするとこれが出てくる

仕様書を読もう！



Introduction

Structure

Validation

Execution

Binary Format

- Conventions
- Values
- Types
- Instructions

Modules

The encoding of a **module** starts with a preamble containing a 4-byte magic number (the string ‘\0asm’) and a version field. The current version of the WebAssembly binary format is 1.

The preamble is followed by a sequence of **sections**. **Custom sections** may be inserted at any place in this sequence, while other sections must occur at most once and in the prescribed order. All sections can be empty.

The lengths of lists produced by the (possibly empty) **function** and **code** section must match up.

Similarly, the optional data count must match the length of the **data segment** list. Furthermore, it must be present if any **data index** occurs in the code section.

```
magic ::= 0x00 0x61 0x73 0x6D
version ::= 0x01 0x00 0x00 0x00
module ::= magic version
         customsec* type*:typesec
         customsec* import*:importsec
```

← **magic, version** は
ここで定義されている ⇒

下までスクロールするとこれが出てくる

magic とは

マジックナンバーのこと

- **ファイルがどのような種類なのかを示す**特定の数字
 - file コマンドはこれを見てファイルの種類を判定することが多い
- たいいていファイルの冒頭にある

マジックナンバーの例：PNG

Additional information:

Deprecated alias names for this type:

N/A

Magic number(s):

89 50 4E 47 0D 0A 1A 0A

File extension(s):

.png

Macintosh file type code:

PNGf

W3C による PNG の仕様書より

マジックナンバーの例：PNG

```
$ file image.png
```

```
image.png: PNG image data, 1920 x 1080, 8-bit/color  
RGB, non-interlaced
```

```
$ od -tx1 image.png | head -n3
```

```
00000000 89 50 4e 47 0d 0a 1a 0a 00 00 00 0d 49 48 44 52  
00000020 00 00 07 80 00 00 04 38 08 02 00 00 00 67 b1 56  
00000040 14 00 00 00 20 63 48 52 4d 00 00 7a 26 00 00 80
```

結論

00000000 00 61 73 6d 0d 00 01 00 07 24 01 42 04 01 6f 02

- マジックナンバーが最初の 4 バイト
- 差異が発生していたその後の 4 バイトはバージョン⁶

⁶ 厳密には違う（参考：スライド最後の補足資料）

目次

1. WebAssembly とは
2. Wasm テキスト形式
3. もっと Wasm を書いてみる
4. Wasm をブラウザで実行してみよう

もっと Wasm を書いてみる

やっぱり**関数**があるとうれしいですね

1. 左側のエディタに以下を入力：

```
(module  
  (func)  
)
```

3. Ctrl+Enter もしくは右上の「Parse」ボタンを押す

waspl

1 (module

2 | (func)

3)

4

24 bytes

Save .wasm

Parse (Ctrl+Enter)

Offset	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00000000	00	61	73	6D	01	00	00	00	01	04	01	60	00	00	03	02
00000010	01	00	0A	04	01	02	00	0B								
	Header Type Function Code															

出力が増えた！

もっと Wasm を書いてみる

(func)を 2 つにしてみると……？

```
(module  
  (func)  
  (func)  
)
```

waspl

Save .wasm Parse (Ctrl+Enter)

```

1  (module
2    (func)
3  )
4

```

24 bytes

Offset	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00000000	00	61	73	6D	01	00	00	00	01	04	01	60	00	00	03	02
00000010	01	00	0A	04	01	02	00	0B								

Header Type Function Code



waspl

Save .wasm Parse (Ctrl+Enter)

```

1  (module
2    (func)
3    (func)
4  )
5

```

28 bytes

Offset	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00000000	00	61	73	6D	01	00	00	00	01	04	01	60	00	00	03	03
00000010	02	00	00	0A	07	02	02	00	0B	02	00	0B				

Header Type Function Code

もっと増えた！

Wasm バイナリのしくみ

Wasm バイナリは**セクション**という塊の集合体

```

magic ::= 0x00 0x61 0x73 0x6D
version ::= 0x01 0x00 0x00 0x00
module ::= magic version
customsec* type*:typesec
customsec* import*:importsec
customsec* typeidx*:funcsec
customsec* table*:tablesec
customsec* mem*:memsec
customsec* tag*:tagsec
customsec* global*:globalsec
customsec* export*:exportsec
customsec* start?:startsec
customsec* elem*:elemsec
customsec* n?:datacntsec
customsec* (local*, expr)*:codesec
customsec* data*:datasec
customsec*

module type* import* tag* global* mem* table* func* data* elem* start? export*
if (n = |data*|)?
 $\wedge (n? \neq \epsilon \vee \text{dataidx}(func*) = \epsilon)$ 
 $\wedge (func = func\ typeidx\ local* expr)*$ 

```

⇒

セクションとは？

次のようなバイト列⁷：

$$\text{section}_N(X) ::= N : \text{byte} \quad \text{len} : \text{u32} \quad \text{content}^* : X \Rightarrow \text{content}^*$$

つまり、

1. セクション ID
2. セクションの中身の長さ
3. セクションの中身

という順に置かれる

⁷ <https://webassembly.github.io/spec/core/binary/modules.html#sections>

セクション ID?

仕様書に書いてある：

The following section ids are used:

Id	Section
0	custom section
1	type section
2	import section
3	function section
4	table section
5	memory section
6	global section
7	export section
8	start section
9	element section
10	code section
11	data section
12	data count section
13	tag section

確認してみよう

コードをこう変えてみる

```
(module
  (func
    (func (export "sample") (result i32)
      i32.const 12345
    )
  )
)
```

Wasm バイナリをダウンロードして

`wasm-tools dump module.wasm` を実行

waspl

48 bytes

Save .wasm

Parse (Ctrl+Enter)

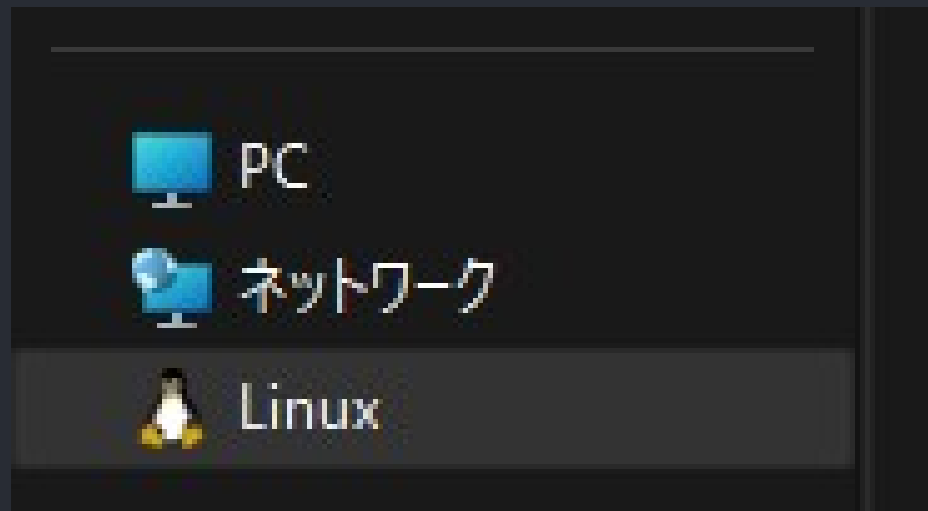
Offset	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00000000	00	61	73	6D	01	00	00	00	01	08	02	60	00	00	60	00
00000010	01	7F	03	03	02	00	01	07	0A	01	06	73	61	6D	70	6C
00000020	65	00	01	0A	0B	02	02	00	0B	06	00	41	B9	E0	00	0B

Header	Type	Function	Export	Code
--------	------	----------	--------	------

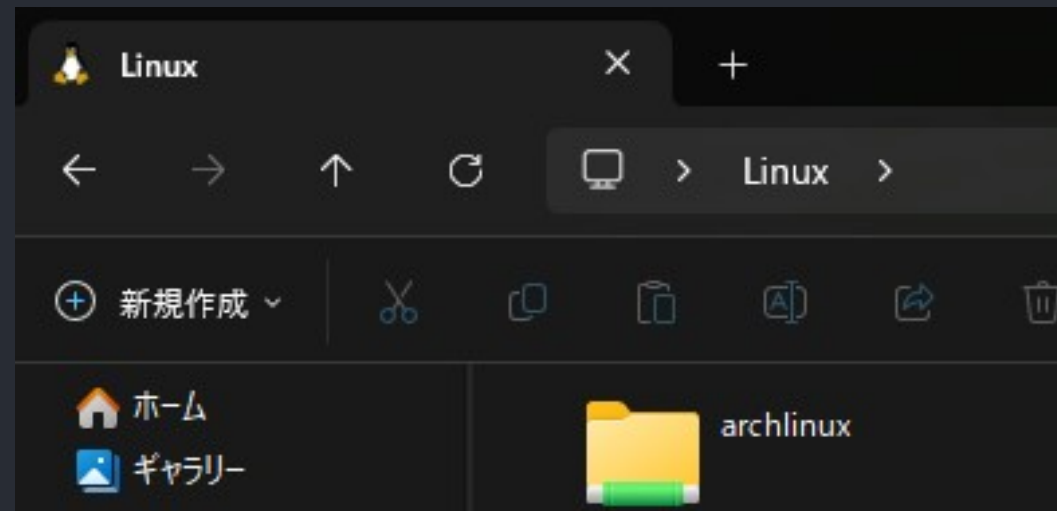
やりかた (Windows 11+WSL)

エクスプローラーで「Linux」を開く

“archlinux”の中にある“root”フォルダに Wasm を保存



エクスプローラ左下あたり



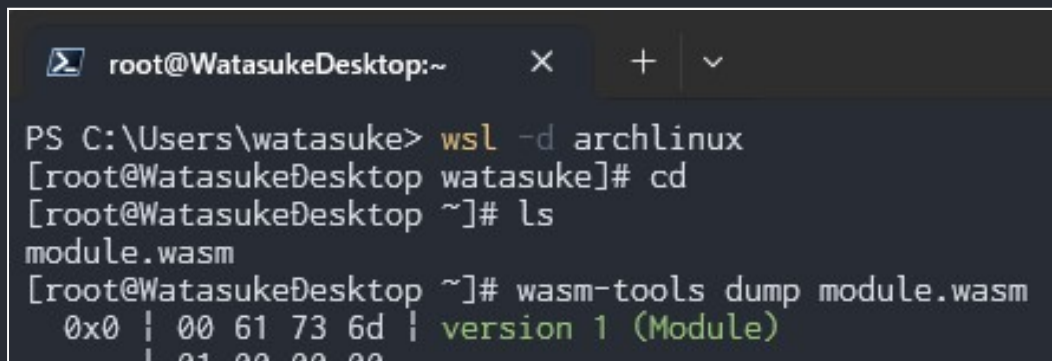
“Linux” をクリックするようになる

やりかた (Windows 11+WSL)

“ターミナルに入力→Enter” を 1 行ずつやる：

```
cd  
wasm-tools dump [ダウンロードしたファイル名]
```

※ターミナルを 1 度閉じていた場合、最初に
wsl -d archlinux を実行すること



```
root@WatasukeDesktop:~  
PS C:\Users\watasuke> wsl -d archlinux  
[root@WatasukeDesktop watasuke]# cd  
[root@WatasukeDesktop ~]# ls  
module.wasm  
[root@WatasukeDesktop ~]# wasm-tools dump module.wasm  
0x0 | 00 61 73 6d | version 1 (Module)  
| 01 00 00 00
```

← 実行例

実行結果

```
% wasm-tools dump module.wasm
0x0 | 00 61 73 6d | version 1 (Module)
    | 01 00 00 00
0x8 | 01 08       | type section
0xa | 02          | 2 count
--- rec group 0 (implicit) ---
0xb | 60 00 00    | [type 0] SubType { is_final: true, supertype_idx: None, c
riptor_idx: None, describes_idx: None } }
--- rec group 1 (implicit) ---
0xe | 60 00 01 7f | [type 1] SubType { is_final: true, supertype_idx: None, c
escriptor_idx: None, describes_idx: None } }
0x12 | 03 03       | func section
0x14 | 02          | 2 count
0x15 | 00          | [func 0] type 0
0x16 | 01          | [func 1] type 1
0x17 | 07 0a       | export section
0x19 | 01          | 1 count
0x1a | 06 73 61 6d | export Export { name: "sample", kind: Func, index: 1 }
    | 70 6c 65 00
    | 01
0x23 | 0a 0b       | code section
0x25 | 02          | 2 count
===== func 0 =====
0x26 | 02          | size of function
0x27 | 00          | 0 local blocks
0x28 | 0b          | end
===== func 1 =====
0x29 | 06          | size of function
0x2a | 00          | 0 local blocks
0x2b | 41 b9 e0 00 | i32_const value:12345
0x2f | 0b          | end
```

セクション

現在のセクションは主に 4 つ：

1. **Type** セクション
2. **Function** セクション
2. **Export** セクション
3. **Code** セクション

```
1  (module
2    (func
3      (func (export "sample") (result i32)
4        i32.const 12345
5      )
6  )
```

Offset	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
00000000	00	61	73	6D	01	00	00	00	01	08	02	60	00	00	60	00
00000010	01	7F	03	03	02	00	01	07	0A	01	06	73	61	6D	70	6C
00000020	65	00	01	0A	0B	02	02	00	0B	06	00	41	B9	E0	00	0B
Header	<u>Type</u>				<u>Function</u>				<u>Export</u>				<u>Code</u>			

Type セクション

- id: 1
- どのような関数が含まれているか伝えるもの

```
0x8 | 01 08 | type section
0xa | 02 | 2 count
--- rec group 0 (implicit) ---
0xb | 60 00 00 | [type 0] SubType { is_final:
ripter_idx: None, describes_idx: None } }
--- rec group 1 (implicit) ---
0xe | 60 00 01 7f | [type 1] SubType { is_final:
escriptor_idx: None, describes_idx: None } }
```

Function セクション

- id: 3
- N 番目の関数の種類が、Type セクションで定義されたタイプのどれに対応するか指定する

0x12		03 03		func section
0x14		02		2 count
0x15		00		[func 0] type 0
0x16		01		[func 1] type 1

Export セクション

- id: 7
- エクスポートする（外から使えるようにする）
関数などを指定する
- 外から見える名前も一緒に指定する

0x17		07 0a		export section
0x19		01		1 count
0x1a		06 73 61 6d		export Export { name: "sample", kind: Func, index: 1 }
		70 6c 65 00		
		01		

Code セクション

- id: 10 (0x0a)
- 関数の中身を格納する

```
0x23 | 0a 0b | code section
0x25 | 02 | 2 count
===== func 0 =====
0x26 | 02 | size of function
0x27 | 00 | 0 local blocks
0x28 | 0b | end
===== func 1 =====
0x29 | 06 | size of function
0x2a | 00 | 0 local blocks
0x2b | 41 b9 e0 00 | i32_const value:12345
0x2f | 0b | end
```

waspl で見てみよう

右側のバイナリにカーソルを乗せると詳細を確認できる

Offset	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
000000000	00	61	73	6D	01	00	00	00	01	04	01	60	00	00	03	02
000000010	01	00	0A	04	01	02	00	0B								

Header Type Function Code

Type

Type section id

0x000000008-0x000000008

関数の中身とは？

スタックマシンにて実行される命令のリスト

スタックマシン

スタックと呼ばれる**データ構造**がある

- 最初に入れた要素が最後に取り出されるような構造
 - このため **FILO** (First-In Last-Out) とも呼ぶ
- お皿を重ねる様子と同じ
 - 上に乗っているお皿を取り出さないと、一番下にあるお皿を取り出せない

スタックマシンの挙動

```
(func (result i32)
  i32.const 1
  i32.const 2
  i32.add
  i32.const 0
  i32.mul
)
```



スタック

スタックマシンの挙動

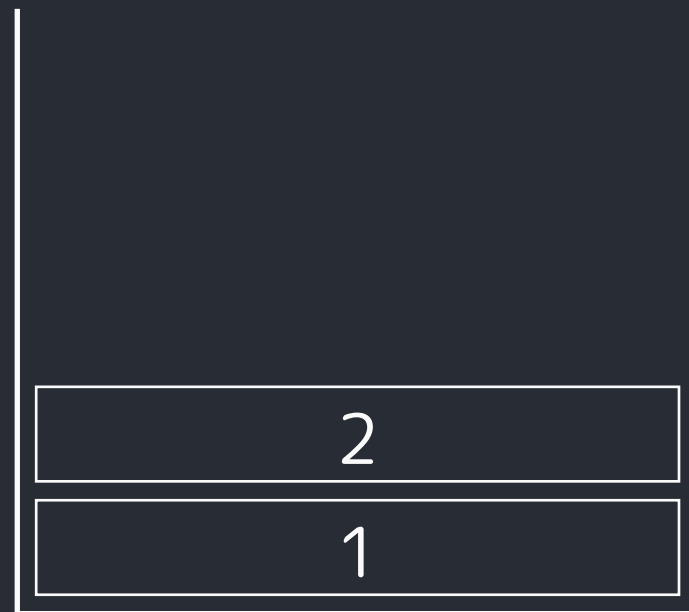
```
(func (result i32)  
  i32.const 1  
  i32.const 2  
  i32.add  
  i32.const 0  
  i32.mul  
)
```



スタック

スタックマシンの挙動

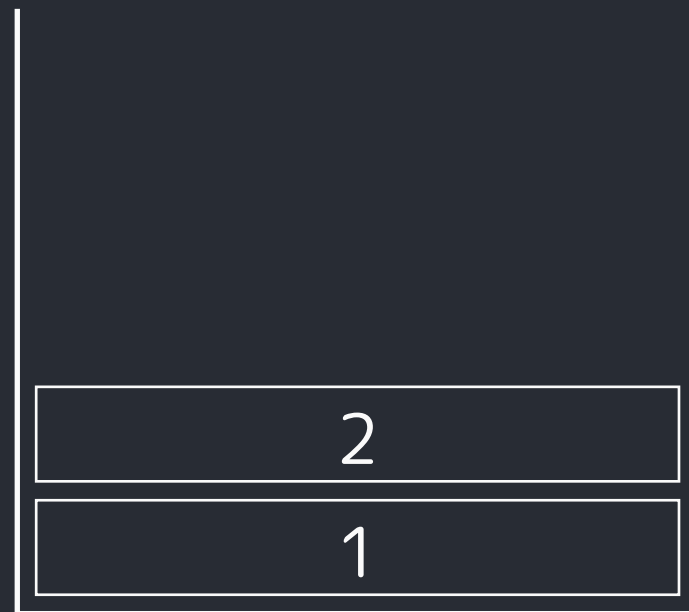
```
(func (result i32)
  i32.const 1
  i32.const 2
  i32.add
  i32.const 0
  i32.mul
)
```



スタック

スタックマシンの挙動

```
(func (result i32)
  i32.const 1
  i32.const 2
  i32.add
  i32.const 0
  i32.mul
)
```



スタック

スタックマシンの挙動

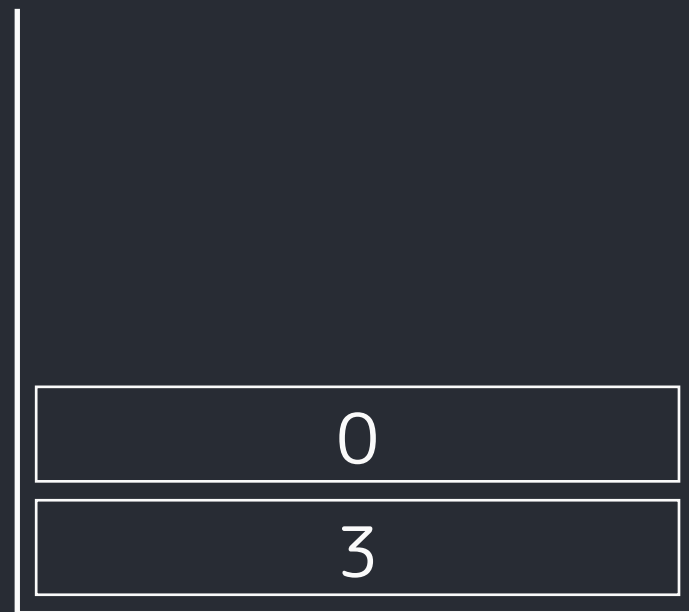
```
(func (result i32)
  i32.const 1
  i32.const 2
  i32.add
  i32.const 0
  i32.mul
)
```



スタック

スタックマシンの挙動

```
(func (result i32)
  i32.const 1
  i32.const 2
  i32.add
  i32.const 0
  i32.mul
)
```



スタック

スタックマシンの挙動

```
(func (result i32)
  i32.const 1
  i32.const 2
  i32.add
  i32.const 0
  i32.mul
)
```



スタック

関数の中身とは？

スタックマシンにて実行される**命令**のリスト

i32.const

スタックに定数を積む

i32.add/sub/mul/div_u

スタックから 2 つ値を取り出して
四則演算した結果をスタックに積む

i32.eq

スタックから 2 つ値を取り出して
同じ値なら 1、違う値なら 0 を
スタックに積む

drop

スタックから値を取り出して、捨てる

WebAssembly Opcodes by Pengo Wray

Reviewed September 2026 | [About](#)

ARRANGE



Grid



Cards



List

Show ▾

MAP ^

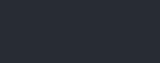
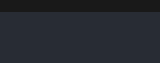
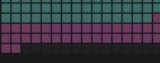
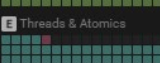
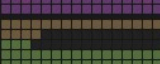
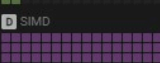
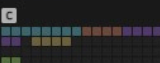


Table 0 201 opcodes

Single-byte instructions.

WebAssembly 1.0

Exception handling

Tail call

Typed function references

Reference types

Sign-extension operators

Garbage collection

Stack switching

Opcode prefix

0x	_0	_1	_2	_3	_4	_5	_6	_7	_8	_9	_A	_B	_C	_D	_E	_F
0_	unreachable	nop	block	loop	if	else			throw		throw_ref	end	br	br_if	br_table	return
1_	call	call_indirect	return_call	return_call_indirect	call_ref	return_call_ref					drop	select	select_t			try_table
2_	local.get	local.set	local.tee	global.get	global.set	table.get	table.set		i32.load	i64.load	f32.load	f64.load	i32.load8_s	i32.load8_u	i32.load16_s	i32.load16_u
3_	i64.load8_s	i64.load8_u	i64.load16_s	i64.load16_u	i64.load32_s	i64.load32_u	i32.store	i64.store	f32.store	f64.store	i32.store8	i32.store16	i64.store8	i64.store16	i64.store32	memory.size
4_	memory.grow	i32.const	i64.const	f32.const	f64.const	i32.eqz	i32.eq	i32.ne	i32.lt_s	i32.lt_u	i32.gt_s	i32.gt_u	i32.le_s	i32.le_u	i32.ge_s	i32.ge_u
5_	i64.eqz	i64.eq	i64.ne	i64.lt_s	i64.lt_u	i64.gt_s	i64.gt_u	i64.le_s	i64.le_u	i64.ge_s	i64.ge_u	f32.eq	f32.ne	f32.lt	f32.gt	f32.le
6_	f32.ge	f64.eq	f64.ne	f64.lt	f64.gt	f64.le	f64.ge	i32.clz	i32.ctz	i32.popcnt	i32.add	i32.sub	i32.mul	i32.div_s	i32.div_u	i32.rem_s
7_	i32.rem_u	i32.and	i32.or	i32.xor	i32.shl	i32.shr_s	i32.shr_u	i32.rotl	i32.rotr	i64.clz	i64.ctz	i64.popcnt	i64.add	i64.sub	i64.mul	i64.div_s

WebAssembly Opcodes (非公式)

さっきのコードの意味は？

```
(func (export "sample") (result i32)
  i32.const 12345
)
```

さっきのコードの意味は？

"sample" という名前で関数をエクスポート

```
(func (export "sample") (result i32)
  i32.const 12345
)
```

さっきのコードの意味は？

戻り値は i32 (32bit 整数)

```
(func (export "sample") (result i32)  
  i32.const 12345  
)
```

さっきのコードの意味は？

```
(func (export "sample") (result i32)  
  i32.const 12345  
)
```

スタックに i32 型の定数 12345 を積む

注意

- Wasm の整数型に**符号付き・符号なし**という意味はない
 - Rust の i32 等と異なる
- Wasm は**命令でこれらを区別する**
 - ある変数を符号付きとして扱いたい場合は
_s (**S**igned) と付く命令を使う
 - 符号なしとして扱いたいなら
_u (**U**nsigned) と付く命令を使う

関数

- (result ...)と書くと、戻り値の型を指定できる
- 複数の値を返すことも出来る
 - (result i32 i32) は i32 を 2 つ返す
- 関数終了時、**スタックに残っていた値が戻り値**となる

関数

- スタックに残っている値は**戻り値のみ**でなければならない
 - 余分な値が残るようなコードを書いてはいけない
- 本資料では**コンパイルできないコードが登場します**
(スペースに余裕がないため)

```
(func (result i32)
  i32.const 0
  i32.const 1
)
```

これはだめ（コンパイルエラーになる）

add 関数を作ってみよう

「引数として数字を 2 つ受け取って、それらを足した数を返す」
という関数を作ってください

→ WebAssembly でやってみよう！

```
fn add(lhs: i32, rhs: i32) -> i32 {  
    lhs + rhs  
}
```

"add"という名前でエクスポートしよう

2つの引数を受け取るには？

- (param i32 i32) を使う
- その後、local.get N を用いて引数をスタックに積む
 - 1つ目の引数は local.get 0 で取得 (0-indexed)

```
(func $f (param i32 i32) (result i32)
  local.get 0)
(func (result i32)
  i32.const 100
  i32.const 200
  call $f ;; return 100
)
```

こたえ

次のページにあります（ネタバレ防止）

```
(module  
  (func (export "add") (param i32 i32) (result i32)  
    local.get 0  
    local.get 1  
    i32.add  
  )  
)
```

そういえば

2つの引数を受け取る例：

```
(func $f (param i32 i32) (result i32)
  local.get 0
)
(func (result i32)
  i32.const 100
  i32.const 200
  call $f ;; return 100
)
```

そういえば

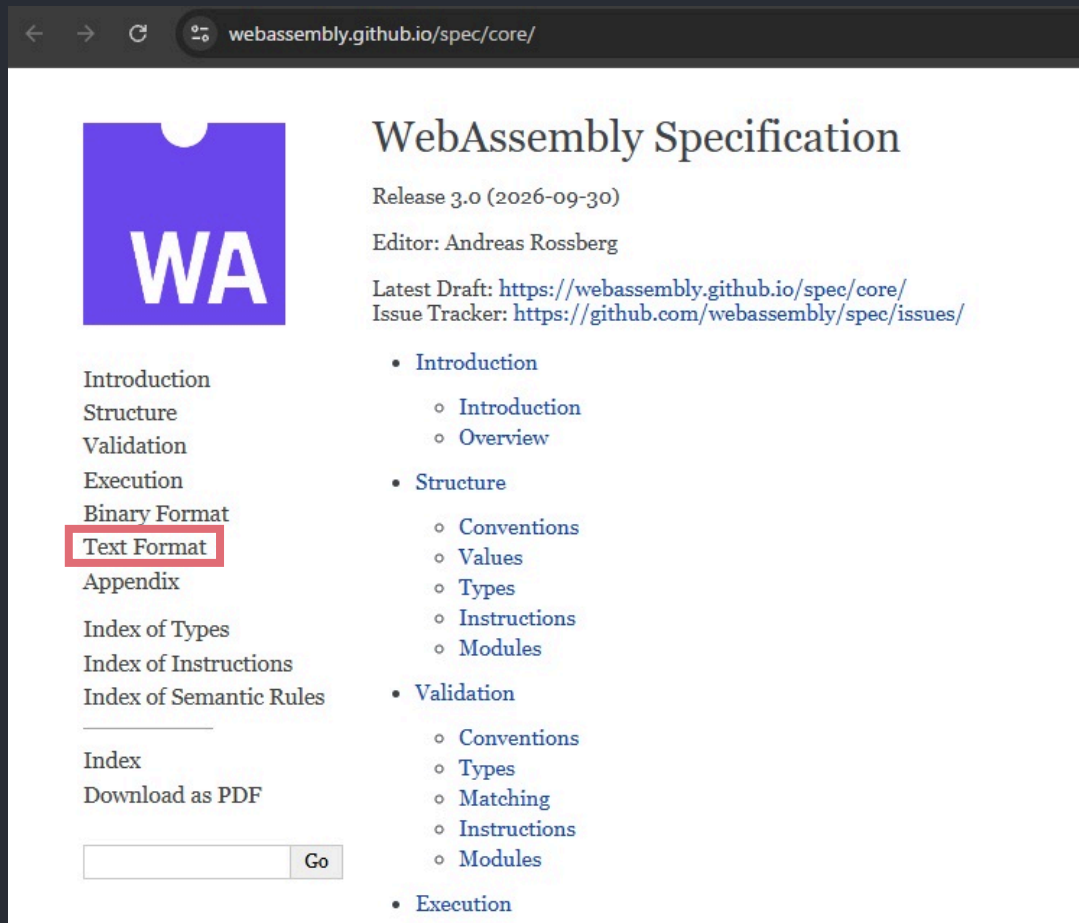
2つの引数を受け取る例：

```
(func $f (param i32 i32) (result i32)
  local.get 0
)
(func (result i32)
  i32.const 100
  i32.const 200
  call $f ;; return 100
)
```

しれっと関数名を定義していた

そういえば

114



The screenshot shows the WebAssembly Specification website. The browser address bar displays 'webassembly.github.io/spec/core/'. The page features a purple 'WA' logo on the left. A sidebar on the left lists navigation links: Introduction, Structure, Validation, Execution, Binary Format, Text Format (highlighted with a red box), Appendix, Index of Types, Index of Instructions, Index of Semantic Rules, Index, and Download as PDF. The main content area is titled 'WebAssembly Specification' and includes release information (Release 3.0 (2026-09-30)), editor information (Editor: Andreas Rossberg), and links for the latest draft and issue tracker. A table of contents on the right lists sections: Introduction (Introduction, Overview), Structure (Conventions, Values, Types, Instructions, Modules), Validation (Conventions, Types, Matching, Instructions, Modules), and Execution.

webassembly.github.io/spec/core/

WebAssembly Specification

Release 3.0 (2026-09-30)

Editor: Andreas Rossberg

Latest Draft: <https://webassembly.github.io/spec/core/>
Issue Tracker: <https://github.com/webassembly/spec/issues/>

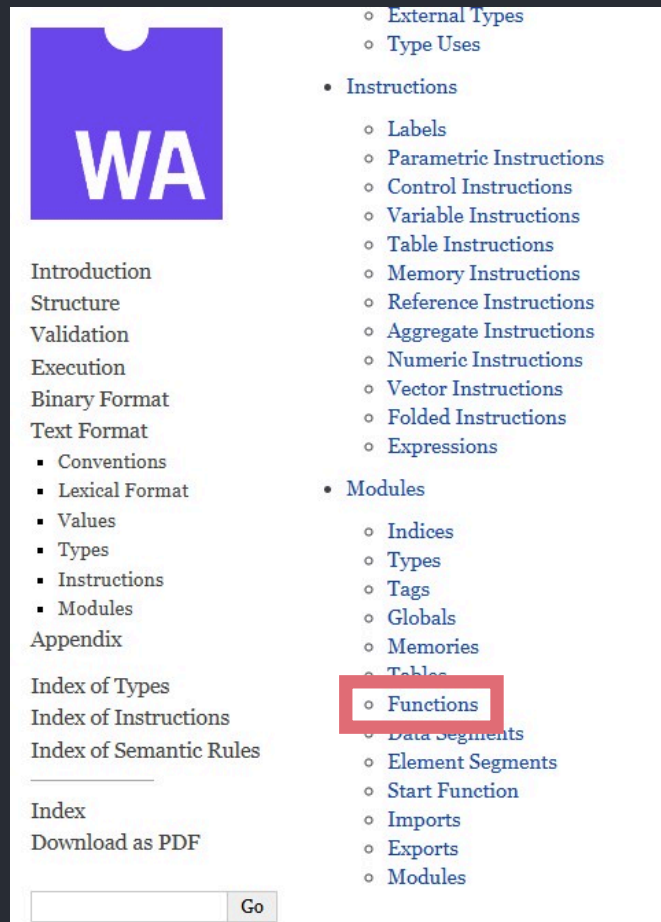
- Introduction
 - Introduction
 - Overview
- Structure
 - Conventions
 - Values
 - Types
 - Instructions
 - Modules
- Validation
 - Conventions
 - Types
 - Matching
 - Instructions
 - Modules
- Execution

Introduction
Structure
Validation
Execution
Binary Format
Text Format
Appendix
Index of Types
Index of Instructions
Index of Semantic Rules
Index
Download as PDF

Go

Text Format もあったな

仕様書を見る



下までスクロールして“Functions”

仕様書を見える

Functions

Function definitions can bind a symbolic **function** identifier, and **local identifiers** for its **parameters** and **locals**.

$$\begin{aligned} \text{func}_I &::= \text{'(' 'func' } id^?:id^? (x, I_1):typeuse_I ((loc^*, I_2):local_I)^* c:expr_I, \text{'')} \Rightarrow \\ &\quad (\text{func } x (\oplus loc^{**}) e, \{\text{funcs } (id^?)\}) \\ &\quad \text{if } I' = I \oplus I_1 \oplus \oplus I_2^* \\ &\quad \wedge \vdash I' : \text{ok} \end{aligned}$$

$$\begin{aligned} \text{local}_I &::= \text{'(' 'local' } id^?:id^? t:valtype_I \text{'')} \Rightarrow \\ &\quad (\text{local } t, \{\text{locals } (id^?)\}) \end{aligned}$$

Note:

The well-formedness condition on I' ensures that parameters and locals do not contain duplicate identifiers.

Abbreviations

Multiple anonymous locals may be combined into a single declaration:

$$\text{local}_I ::= \dots \mid \text{'(' 'local' } valtype_I^* \text{'')} \equiv (\text{'(' 'local' } valtype_I \text{'')})^*$$

Functions can be defined as **imports** or **exports** inline:

$$\begin{aligned} \text{import}_I &::= \dots \\ &\quad \mid \text{'(' 'func' } id^? \text{'(' 'import' } name^2 \text{'')} typeuse_I \text{'')} \equiv \\ &\quad \text{'(' 'import' } name^2 \text{'(' 'func' } id^? typeuse_I \text{'')} \text{'')} \\ \text{export}_I &::= \text{'(' 'func' } id^?:id^? \text{'(' 'export' } name \text{'')} \dots \text{'')} \equiv \\ &\quad \text{'(' 'func' } id^?:id \text{'(' } \dots \text{'')} \\ &\quad \text{'(' 'export' } name \text{'(' 'func' } id \text{'')} \text{'')} \\ &\quad \text{if } id^? = id' \vee id^? = \epsilon \wedge id' \notin I.\text{funcs} \end{aligned}$$

仕様書を見える

Functions

Function definitions can bind a symbolic **function identifier**, and **local identifiers** for its parameters and

$$\begin{aligned} \text{func}_I &::= \text{'(' 'func' } id^?:id^? (x, I_1):\text{typeuse}_I ((loc^*, I_2):\text{local}_I)^* e:\text{expr}_{I'} \text{'})' \\ &\quad (\text{func } x (\bigoplus loc^{**}) e, \{\text{funcs } (id^?)\}) \\ &\quad \text{if } I' = I \oplus I_1 \oplus \bigoplus I_2^* \\ &\quad \wedge \vdash I' : \text{ok} \end{aligned}$$

'(' 'func' の後に来るもの → id というらしい
? が付いている == 省略可能という意味

仕様書を見える

- `id` はコード内で参照するためのもの
 - あってもなくても出力される Wasm は変わらない
 - `export` 時の名前とは関係がない
- `?` が付いている == 省略可能という意味

Functions

Function definitions can bind a symbolic `function identifier`, and `local identifiers` for its parameters and

$$\text{func}_I ::= \text{'(' 'func' } id^?:id^? (x, I_1):\text{typeuse}_I ((loc^*, I_2):\text{local}_I)^* c:\text{expr}_I, \text{'})'}$$

$$(\text{func } x (\bigoplus loc^{**}) c, \{\text{funcs } (id^?)\})$$

ちなみに

さっきの add の例は……

```
(module
  (func (export "add") (param i32 i32) (result i32)
    local.get 0
    local.get 1
    i32.add
  )
)
```

ちなみに

こうも書けます

```
(module
  (func (param i32 i32) (result i32)
    local.get 0
    local.get 1
    i32.add
  )
  (export "add" (func 0))
)
```

ということなの

```
(func (export "add") (param i32 i32) (result i32)
  ;; ...
)
```

```
(func (param i32 i32) (result i32)
  ;; ...
)
(export "add" (func 0))
```

仕様書を見してみる

Abbreviations

Multiple anonymous locals may be combined into a single declaration:

$$\text{local}_I ::= \dots \mid ' (' \text{'local' valtype}_I ')' \equiv ((' (' \text{'local' valtype}_I ')')^*$$

Functions can be defined as `imports` or `exports` inline:

$$\begin{aligned} \text{import}_I &::= \dots \\ &\mid ' (' \text{'func' id}^? ' (' \text{'import' name}^2 ') \text{typeuse}_I ')' \equiv \\ &\quad ' (' \text{'import' name}^2 ' (' \text{'func' id}^? \text{typeuse}_I ') ')' \\ \text{export}_I &::= ' (' \text{'func' id}^?:\text{id}^? ' (' \text{'export' name} ') \dots ')' \equiv \end{aligned}$$

Abbreviation (省略) だからそれっぽい

仕様書を見える

Abbreviations

Multiple anonymous locals may be combined into a single declaration:

```
localI ::= ... | '(' 'local' valtypeI ')' ≡ ((' 'local' valtypeI ')')*
```

Functions can be defined as imports or exports inline:

```
importI ::= ...
           | '(' 'func' id? '(' 'import' name2 ')' typeuseI ')' ≡
             '(' 'import' name2 '(' 'func' id? typeuseI ')' ')'
exportI ::= '(' 'func' id?:id? '(' 'export' name ')' ... ')' ≡
```

Abbreviation (省略) だからそれっぽい

ちなみに その 2

さっきの add の例は……

```
(module  
  (func (export "add") (param i32 i32) (result i32)  
    local.get 0  
    local.get 1  
    i32.add  
  )  
)
```

ちなみに その 2

こうも書けます

```
(module
  (func (export "add")
    (param $lhs i32) (param $rhs i32) (result i32)
    local.get $lhs
    local.get $rhs
    i32.add
  )
)
```

ちなみに その 2

こうも書けます

```
(module
  (func (export "add")
    (param $lhs i32) (param $rhs i32) (result i32)
    local.get $lhs
    local.get $rhs
    i32.add
  )
)
```

仕様書を見える

Functions

Function definitions can bind a symbolic function identifier, and local identifiers for its parameters and locals.

$$\begin{aligned} \text{func}_I &::= \text{'(' 'func' } id?:id? \boxed{(x, I_1):typeuse_I} ((loc^*, I_2):local_I)^* e:\text{expr}_{I'} \text{'('} \Rightarrow \\ &\quad (\text{func } x (\oplus loc^{**}) e, \{\text{funcs } (id')\}) \\ &\quad \text{if } I' = I \oplus I_1 \oplus \oplus I_2^* \\ &\quad \wedge \vdash I' : \text{ok} \end{aligned}$$

id (関数名) より後、expr (命令) より前、local は使っていない
→ typeuse かな？

仕様書を見える

Type Uses

A *type use* is a reference to a [type definition](#). Where it is required to reference a [function type](#), it may have explicit inlined [parameter](#) and [result](#) declarations. That allows binding symbolic [identifiers](#) to names. If inline declarations are given, then their types must match the referenced [function type](#).

```
typeuseI ::= '(' 'type' x:typeidxI ')'
```

```
if I.typedefs[x] = (rec st*).i
```

```
∧ st*[i] = sub final (func t1* → t2*)
```

```
∧ I' = {locals (ε)|ti*||}
```

(type ...みたいなやつを書いているわけではないので
ちょっと違いそう

仕様書を見してみる

Abbreviations

A type use may also be replaced entirely by inline **parameter** and **result** declarations. In that case, a **type index** is automatically inserted:

```

typeuseI ::= ... | (t1, id?)*:paramI* t2:result tI* ≡ '(' 'type' x:typeidxI ')' par
    if I.typedefs[x] = (rec (sub final (func t1* → t2*))).0
    ∧ (I.typedefs[i] ≠ (rec (sub final (func t1* → t2*))).0)i < x

```

それっぽい（アスタリスク * 付きなので**繰り返し**書ける）

仕様書を見してみる

Composite Types

Composite types are parsed into their respective abstract representation, paired with the local **identifier context** generated by their bound field or parameter identifiers:

<code>comptype_I ::=</code>	<code>(' 'struct' (ft, id[?])*:list(field_I) ')</code>	$\Rightarrow$	<code>(struct ft*, {fields ((id[?])*))}</code>
	<code> (' 'array' ft:fieldtype_I ')</code>	$\Rightarrow$	<code>(array ft, {fields (ε)})</code>
	<code> (' 'func' (t₁, id[?])*:list(param_I) t₂:list(result_I) ')</code>	$\Rightarrow$	<code>(func t₁* → t₂*, {fields (ε)})</code>
<code>field_I ::=</code>	<code>(' 'field' id[?]:id[?] ft:fieldtype_I ')</code>	$\Rightarrow$	<code>(ft, id[?])</code>
<code>param_I ::=</code>	<code>(' 'param' id[?]:id[?] t:valtype_I ')</code>	$\Rightarrow$	<code>(t, id[?])</code>
<code>result_I ::=</code>	<code>(' 'result' t:valtype_I ')</code>	$\Rightarrow$	<code>t</code>
<code>fieldtype_I ::=</code>	<code>zt:storagetype_I</code>	$\Rightarrow$	<code>zt</code>
	<code> (' 'mut' zt:storagetype_I ')</code>	$\Rightarrow$	<code>mut zt</code>

Composit Type かあ

目次

1. WebAssembly とは
2. Wasm テキスト形式
3. もっと Wasm を書いてみる
4. Wasm をブラウザで実行してみよう

Wasm をブラウザで実行してみよう

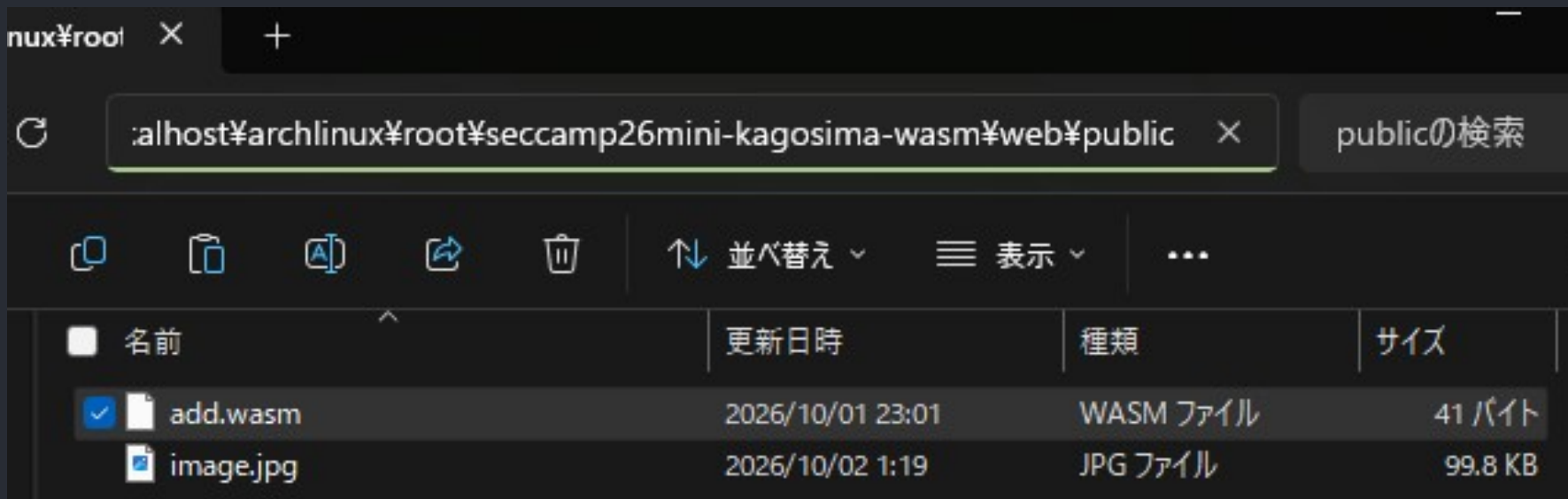
WebAssembly というくらいだからね

先ほど作成した add 関数を呼び出してみよう！

Wasm をブラウザで実行してみよう

1. waspl から Wasm をダウンロード

→ seccamp26mini-kagosima-wasm/web/public/add.wasm
として保存



WSL だとこんな感じになる

Wasm をブラウザで実行してみよう

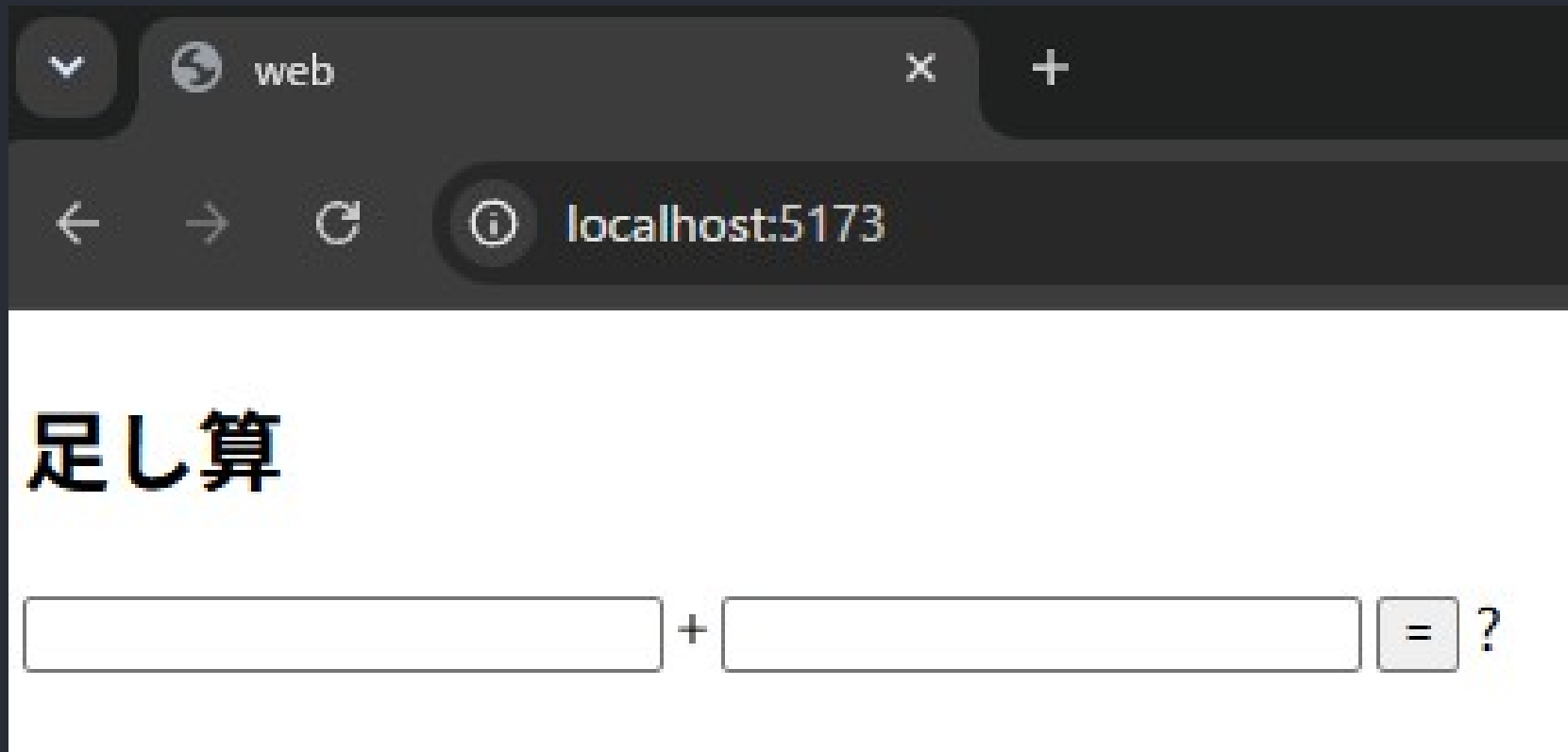
2. “ターミナルに入力→Enter” を 1 行ずつやる :

```
cd ~/seccamp26mini-kagosima-wasm/web  
npm install  
npm run dev
```

※ターミナルを 1 度閉じていた場合、最初に
wsl -d archlinux を実行すること

3. Google Chrome を開いて、URL 欄に
localhost:5173 と入力して Enter

Wasm をブラウザで実行してみよう



こういうページが見えるはず（数字を入れて計算してみよう）

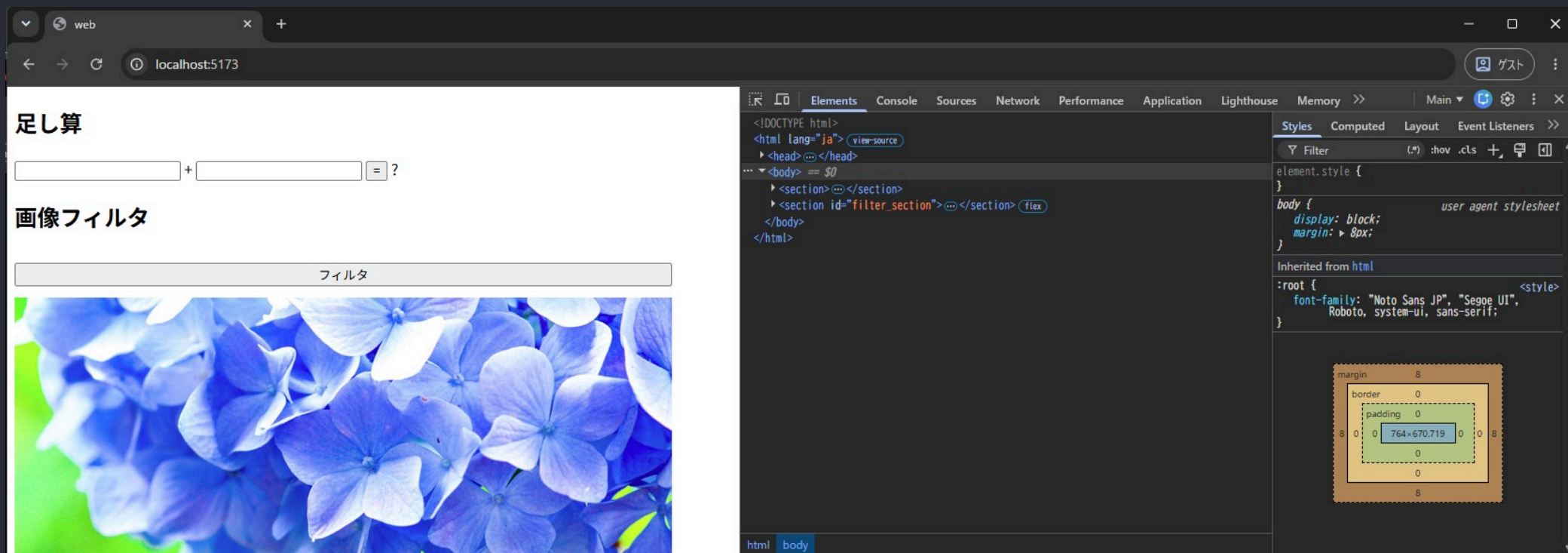
Web 技術

Web ページは基本的に以下の **3 要素**から構成される：

- **HTML** (HyperText Markup Language)
 - 要素およびその構造を定義する
- **CSS** (Cascading Style Sheets)
 - HTML で記述された要素の見た目を変更する
- **JavaScript**
 - HTML 等に動きなどを与える

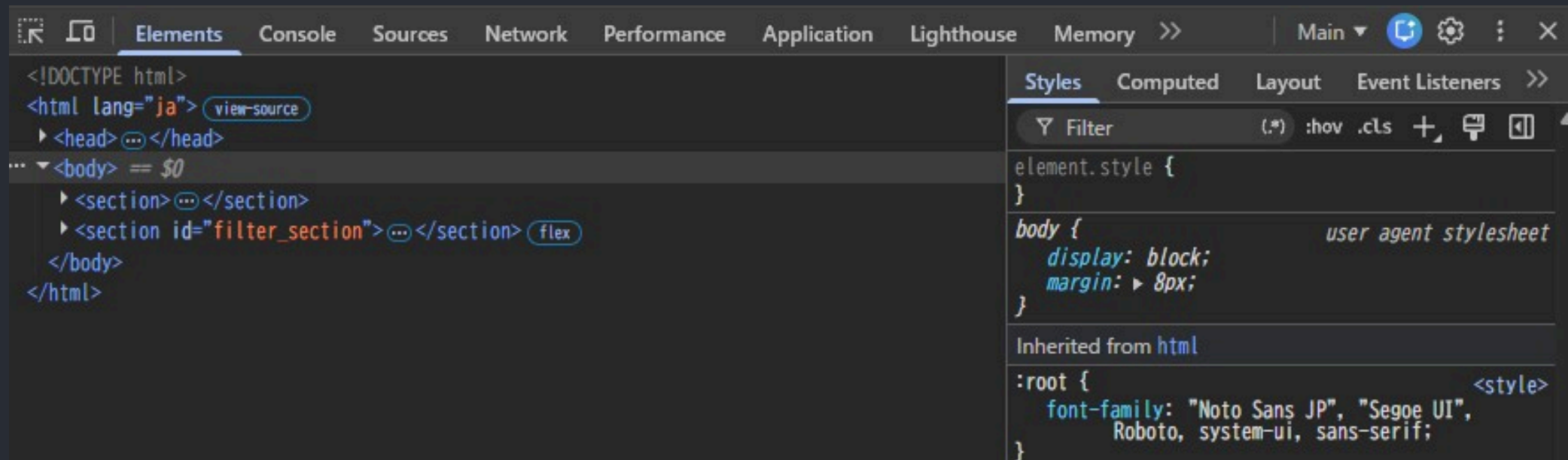
開発者ツールを使ってみよう

F12 キーを押すとこのような画面になる



開発者ツールを使ってみよう

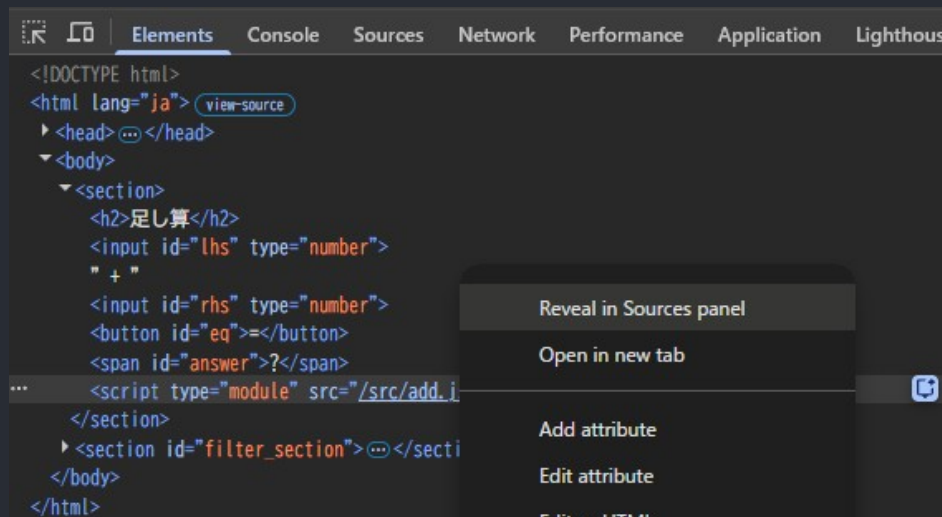
Elements タブでは HTML や CSS のようすを確認できる



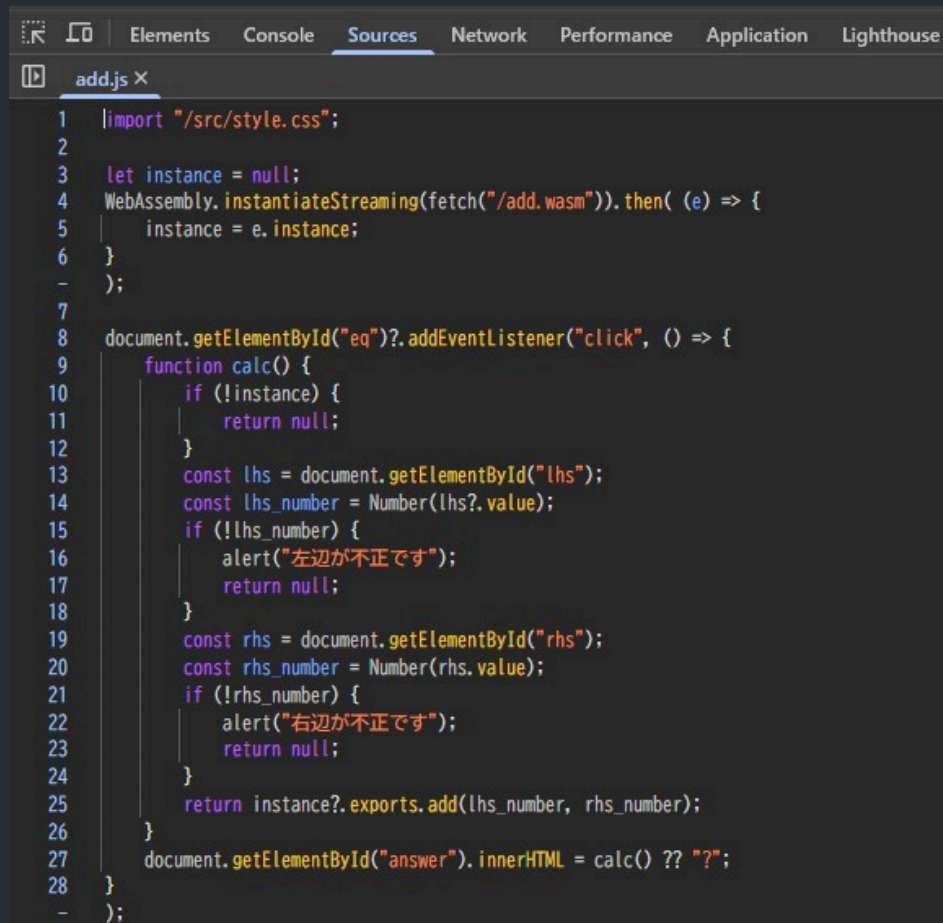
左が HTML、右が CSS

開発者ツールを使ってみよう

- <body>の中にある 2 つの<section>のうち上にあるほうの ► をクリックして展開
- <script type="module" src="/src/add.js"> という行の /src/add.js を右クリックして「Reveal in Sources panel」



開発者ツールを使ってみよう



The screenshot shows the Chrome DevTools Sources panel with the file 'add.js' open. The code is as follows:

```
1  import "/src/style.css";
2
3  let instance = null;
4  WebAssembly.instantiateStreaming(fetch("/add.wasm")).then( (e) => {
5    instance = e.instance;
6  }
7  );
8
9  document.getElementById("eq")?.addEventListener("click", () => {
10    function calc() {
11      if (!instance) {
12        return null;
13      }
14      const lhs = document.getElementById("lhs");
15      const lhs_number = Number(lhs?.value);
16      if (!lhs_number) {
17        alert("左辺が不正です");
18        return null;
19      }
20      const rhs = document.getElementById("rhs");
21      const rhs_number = Number(rhs.value);
22      if (!rhs_number) {
23        alert("右辺が不正です");
24        return null;
25      }
26      return instance?.exports.add(lhs_number, rhs_number);
27    }
28    document.getElementById("answer").innerHTML = calc() ?? "?";
29  });
```

コードが見える！

JavaScript の実装

- 変数 `instance` を定義
- `fetch` でダウンロードした Wasm ファイルを `WebAssembly.instantiateStreaming` に渡して、
Wasm モジュールからインスタンスを作成する

```
let instance = null;  
WebAssembly.instantiateStreaming(  
  fetch("/add.wasm")  
)  
.then((e) => { instance = e.instance; });
```

JavaScript の実装

ID “eq” が与えられたボタンが
クリックされたときに実行する関数を登録

```
document.getElementById("eq").addEventListener("click", () => {  
    // ...  
});
```

JavaScript の実装

- テキストボックスの値を数字に変換
- 先ほど作ったインスタンスの exports に
エクスポートした関数があるので、実行！

```
const lhs = document.getElementById("lhs");  
const lhs_number = Number(lhs?.value);  
const rhs = document.getElementById("rhs");  
const rhs_number = Number(rhs.value);  
return instance?.exports.add(lhs_number, rhs_number);
```

演習：Wasm で画像を加工しよう

- 画像は赤、緑、青 (RGB) の 3 要素により構築される
- **RGB 値 3 つ**を引数として受け取り、加工をするなどして、**RGB 値 3 つ**を戻り値として返す関数を作ろう
- 作った関数は "filter" と名付けてエクスポートする
- Wasm ファイルを以下のように保存：
seccamp26mini-kagosima-wasm/web/public/filter.wasm
- 「フィルタ」ボタンを押すと、フィルタ処理が実行される

演習：Wasm で画像を加工しよう

関数の定義はどうなる？

(答えは次のページ)

演習：Wasm で画像を加工しよう

```
(module
  (func (export "filter")
    (param i32 i32 i32)
    (result i32 i32 i32)
    local.get 0 ;; R値
  )
)
```

演習：Wasm で画像を加工しよう

こう書くと引数の意味がわかりやすくなる

```
(module
  (func (export "filter")
    (param $r i32)
    (param $g i32)
    (param $b i32)
    (result i32 i32 i32)
    local.get $r ;; R値
  )
)
```

演習：Wasm で画像を加工しよう

例えば、画像全体を緑（#00FF00）にするには？

（答えは次のページ）

演習：Wasm で画像を加工しよう

例えば、画像全体を緑（#00FF00）にするには？

```
(module
  (func (export "filter")
    (param i32 i32 i32) (result i32 i32 i32)
    i32.const 0      ;; R
    i32.const 255    ;; G
    i32.const 0      ;; B
  )
)
```

演習：Wasm で画像を加工しよう



実行したときのようす

演習：Wasm で画像を加工しよう

アイデアリスト：

- グレースケール化
- 色反転
- RGB を入れ替えてみる
- **(発展)** 右のピクセルほど明るくなる……？
 - Wasm の関数を呼び出す部分を変える必要がある

演習：Wasm で画像を加工しよう

使えそうな命令

- 足し算 : **i32.add**
- 引き算 : **i32.sub**
- 掛け算 : **i32.mul**
- 割り算 : **i32.div_u**
- **==** : **i32.eq**
- **!=** : **i32.ne**
- $\leq$: **i32.le_u**
- $<$: **i32.lt_u**
- $\geq$: **i32.ge_u**
- $>$: **i32.gt_u**
 - **L**ess/**G**reat+**E**qual/**T**han + **U**nsigned
 - 今回は負の値を扱わない

演習：Wasm で画像を加工しよう

ローカル変数を用いるスタック複製

```
(func (export "filter") (param i32 i32 i32) (result i32 i32 i32)
  (local i32)           ;; ローカル変数は result の後に宣言
  local.get 0           ;; R値
  local.get 1           ;; G値
  local.get 2           ;; B値
  local.get 3           ;; ローカル変数
  i32.const 128
  local.tee 3
  ;; local[3] に128が格納される+スタック最上部に128が残る
)
```

演習：Wasm で画像を加工しよう

if 文

```
(func (export "filter")  
  (param i32 i32 i32) (result i32 i32 i32)  
  local.get 0          ;; R値  
  i32.const 128  
  i32.lt_s              ;; R < 128 なら1がスタックに積まれる  
  if (result i32)  
    i32.const 0          ;; R < 128なら黒にする  
  else  
    i32.const 255        ;; R >= 128 なら赤  
  end  
)
```

おわりに

- WebAssembly のテキスト形式 / バイナリ形式を見てきた
 - Wasm バイナリは**セクション**という単位によって構成される
 - Code セクションに格納されている命令を**スタックマシン**により実行する
- 今回は一部しか見せられなかったが、Wasm が持つ機能や命令は豊富
 - ぜひためしてみてね

補足

next step (時間が余ったらやりたいこと)

- Wasm instantiate 時の importObject
- 説明していないセクション
 - **import, memory**, global, ...
- DevTools の Performance タブで計測
- WASI & Component Model
- `file /usr/bin/ls` に出てきた ELF って何

余談：「バージョンの違い」とは？

マジックナンバー直後が `0d 00 01 00` になっていたのは
`rustc --target wasm32-wasip2 add.rs` として生成した Wasm
→ WASI (Component Model) 用にビルドされている

cf. [Binary.md](#)

```
component ::= <preamble> s*:<section>*           =>
(component flatten(s*))
preamble   ::= <magic> <version> <layer>
magic      ::= 0x00 0x61 0x73 0x6D
version    ::= 0x0d 0x00
layer      ::= 0x01 0x00
```